SHAHEED ZULFIKAR ALI BHUTTO INSTITUTE OF SCIENCE AND TECHNOLOGY
Karachi Campus

DIGITAL WALLET APPLICATION
(Backend & Dashboards)

Course Semester Project Report
Course: Database Systems (Theory & Lab)

Group Members:
Laiba Kanwal (Reg ID: 2480204)
Shahla (Reg ID: 2480221)

Semester & Section: 4 C
Program: BS Software Engineering (BSSE)

GitHub Repository: https://github.com/Laiba-Kanwal-04/digital-wallet-app

Acknowledgement

We would like to express our deepest gratitude to all those who contributed to the successful conceptualization, design, and execution of our Database Systems semester project, the Digital Wallet Application (PakWallet System). We are exceptionally indebted to our theory and lab course instructors for providing us with foundational instruction in relational database design, data consistency modeling, normalization criteria, structural querying languages, and client-server application paradigms.

We acknowledge the technological infrastructure provided by SZABIST Karachi Campus, which allowed us to develop, integrate, and test our local architectural frameworks smoothly. Lastly, we appreciate our peers and family units for providing persistent technical inspiration, QA feedback, and functional motivation during intensive deployment phases.

Contents

Table of Tables

Figure of Figures

Introduction

The modern financial landscape is rapidly shifting away from conventional brick-and-mortar physical cash operations toward micro-payment, software-driven digital ledger accounts. This project introduces the Digital Wallet Application (commercially conceptualized as PakWallet System)—a production-grade, highly secure, fully integrated 3-Tier full-stack Web application architected expressly to facilitate fluid monetary custody, transaction ledger calculations, multi-user asset clearing, and real-time data-driven operational monitoring. Operating over a decoupled architectural pipeline comprising a presentation layout built with standards-compliant HTML5, functional CSS3, and dynamic Vanilla asynchronous JavaScript, the solution establishes secure transactional tunnels through an enterprise ExpressJS API middleware connected to a high-availability cloud-replicated serverless server-backed relational PostgreSQL instance hosted via Neon Tech engine pipelines.

By establishing highly distinct application contexts, the PakWallet environment addresses the interactive paradigms of modern web interactions. For general users, it surfaces clean, unified visual dashboards detailing current financial assets, transaction logs equipped with live client-side regex matching, structured multi-party beneficiary lists, profile managers, and transactional charts populated by ChartJS Canvas libraries. Simultaneously, for managerial operations, it features an exclusive Administrative Board running on specialized token validation security roles. This interface allows complete manipulation, blocking, and auditing of system users, platform asset aggregates, and ledger traffic logs. This setup ensures that standard workflows remain fully decoupled from overarching administrative oversight.

Problem Statement

In current consumer financial tech frameworks, traditional monetary processes remain highly restricted by data segregation, concurrency flaws, and calculation mismatches. Legacy systems or basic spreadsheet tracking tools introduce critical limitations:

- Data Redundant Anomaly Traps: Common wallet systems fail to implement proper relational schemas, causing account details to be repeatedly stored across separate storage rows. This results in data sync errors when customer states change.
- Unprotected Concurrency Interleaving: Simultaneous double-spend attempts or overlapping deposit-transfer calls frequently trigger dirty reads or non-repeatable state changes. These balance inconsistencies compromise financial audit records.
- Cryptographic Verification Absences: Storing authentication hashes or transaction states in plaintext exposes financial databases to exploit attempts and unauthorized ledger edits.
- Opaque Operational Activity Audits: System administrative teams struggle to monitor platform assets or audit unauthorized users due to missing cross-referenced logs. This lack of visibility weakens internal security controls.

Solution Provided by Project

The Digital Wallet Application eliminates these operational vectors by deploying a fully structural relational paradigm backed by modern middleware logic orchestration. The solution introduces the following core mitigations:

- Strict Third Normal Form (3NF) Partitioning: The storage tier maps entities into distinct structures linked by enforced foreign key constraints. This strategy removes field duplicity and blocks update anomalies completely.
- Atomic ACID Transaction Routines: Balance mutations (such as deposits, transfers, and withdrawals) execute through atomic transactional statements. This design ensures that both the sender's balance deduction and the receiver's balance credit succeed or roll back as a single unit.
- Robust Cryptographic Security Layers: High-entropy JSON Web Tokens secure all API endpoints, while one-way bcryptjs salted hashes protect user credentials. This setup prevents database leaks from compromising live customer accounts.
- Interactive Management Visual Dashboards: Advanced analytics modules combine with administrative control routing to provide instant oversight.

Administrators can adjust user block states with full consistency across all tables.

Table 1

Hardware Requirements

The table below specifies the foundational hardware execution requirements for installing, compiling, and executing the platform environment:

Infrastructure Component	Minimum Architecture Specifications
Processor Architecture	Intel Core i3 Eighth-Gen / AMD Ryzen 3 equivalent or higher processing unit.
Memory Volatility (RAM)	4 Gigabytes minimum physical allocation; 8 Gigabytes optimal for background process execution.
Storage Infrastructure	500 Megabytes free non-volatile partition space for compilation caches and modules.
Network Layer Adapter	IEEE 802.11ac wireless interface or Gigabit Ethernet connection for cloud syncing API routes.
Display Matrix Layout	1366 x 768 native pixel density layout interface (optimized via media responsive CSS breakpoints).

Table 2

Software Requirements

The execution stack relies on the following operational runtime systems, server engines, frameworks, and deployment platforms:

Technological Context Category	Selected Environment Software Standard
Host Operating Architecture	Microsoft Windows 10 / 11, Apple macOS Longhorn, or generic Linux Unix kernels.
Application Engine Runtime	Node.js Environment Infrastructure Platform (LTS Version 18.x or greater compiled).
Core Server Middleware	ExpressJS Minimalist Extensible Routing Node Server Pipeline Engine.
Relational Engine System	PostgreSQL Object-Relational Storage Server System Engine Instance.
Cloud Data Persistence	Neon Tech Serverless Asynchronous Edge Distributed Database Cloud Cluster.
User Presentation Layer	HyperText Markup Language 5, Cascading Style Sheets 3, ECMAScript 6+ standard JavaScript.

Engine Visual Graphing	ChartJS Canvas HTML Element Client Data Binding Library API.
Code Compilation System	Microsoft Visual Studio Code IDE / GitHub Codespaces Dev Environment Container.
Endpoint Request Testing	Postman Enterprise API Testing Suite / cURL Terminal Execution Request Piping.
Version Control Infrastructure	Git Decentralized Engine Source Configuration Tracker linked to GitHub Remotes.
Cryptographic Modules	jsonwebtoken (JWT) security routines and bcryptjs blowfish block-salting encryption modules.

Entity Relationship Diagram (ERD)

The data design layout models structural relationships between users, internal ledgers, beneficiaries, and audit records. The primary Users entity acts as the parent anchor for system interactions. It uses an internal flag to distinguish standard customers from administrative security roles. To manage platform currency activity, the Transactions table explicitly records deposits, withdrawals, and peer transfers. This structure relies on strict foreign key references to link transactions directly to their originating and destination accounts. Additionally, specialized intersection entities handle dynamic user-level configurations. For instance, the Beneficiaries entity links recurrent accounts without duplicate entry structures, while the Notifications table records system activity to provide full historical auditability.

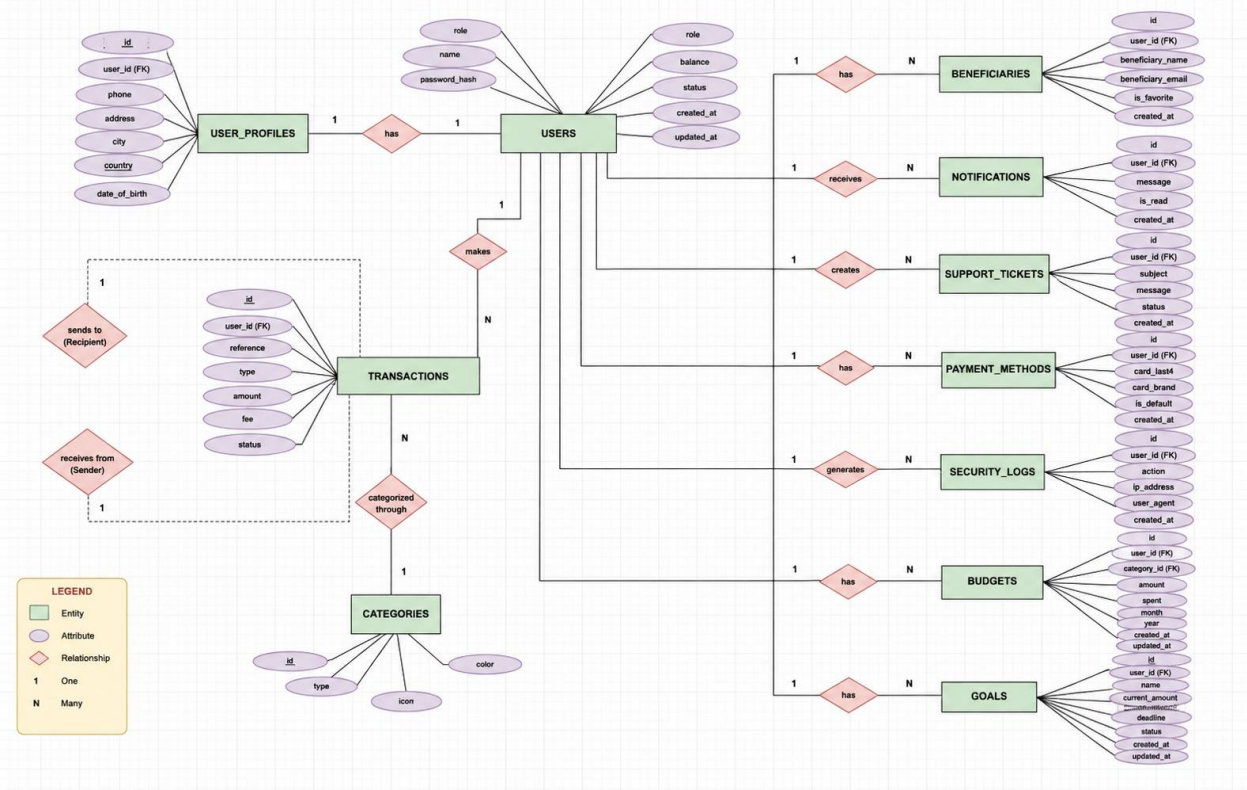


Figure 1

Class Diagram

The Object-Oriented software structural layout encapsulates application functionality into decoupled domain models, middleware filters, and router entry points. The User entity models foundational state properties and signature verification methods, while exposing distinct interfaces for administrative privileges. Financial activities are isolated within specialized class modules, where individual classes manage operations like deposits, transfers, and balance histories. Security boundaries are maintained by an AuthMiddleware class, which intercepts incoming requests to decode tokens before execution. This object model isolates business rules from direct persistence logic, ensuring high modularity and clean test boundaries.

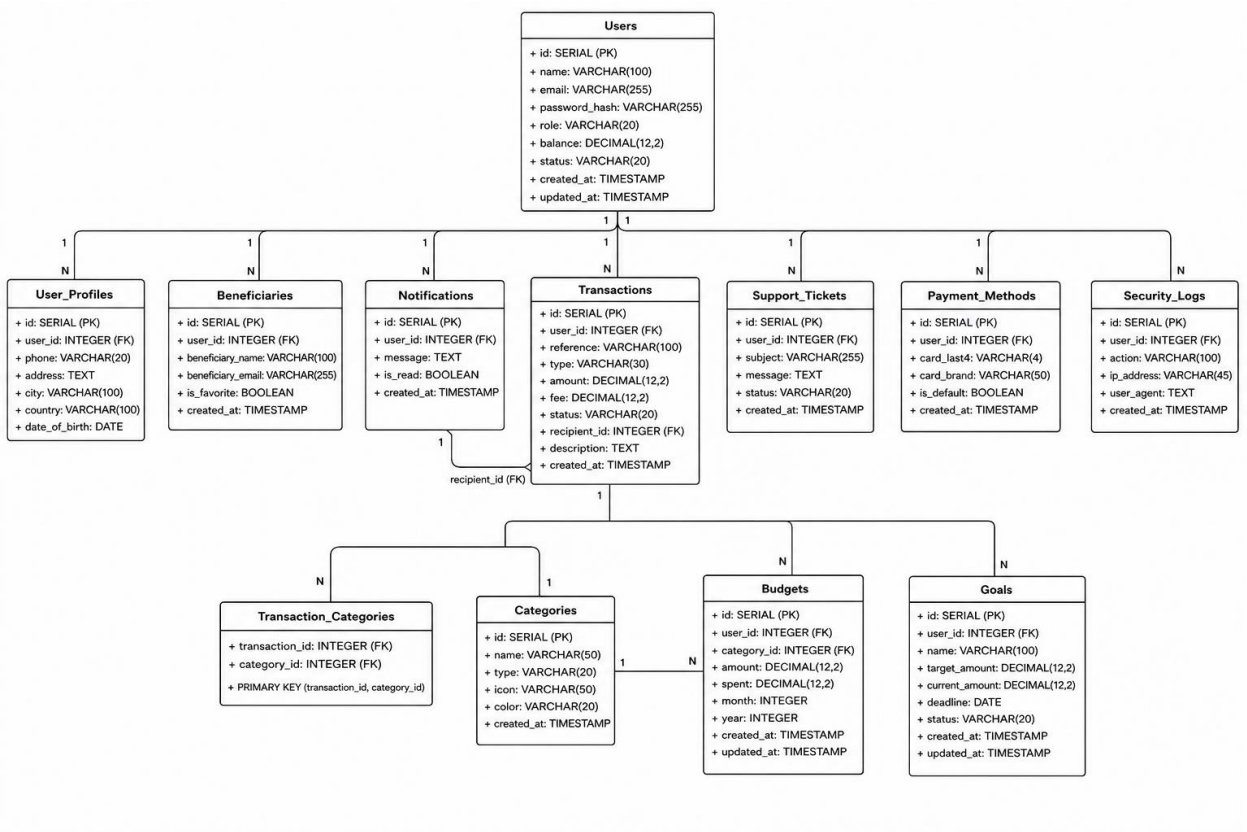


Figure 2

Schema for DBMS (Relational Mapping)

The physical database architecture translates logical entity relations into concrete storage structures. The configuration runs on strict relational constraints, using unique indexes, cascade triggers, and non-nullable fields to protect ledger data from common integrity failures. Below is the mapped layout representation of the active system tables:

1. USERS TABLE (Primary Profile Registry Domain)

[user_id (PK, SERIAL)] -> [name (VARCHAR, NOT NULL)] -> [email (VARCHAR, UNIQUE, INDEX)] -> [password_hash (VARCHAR)] -> [role (VARCHAR, DEFAULT 'user')] -> [status (VARCHAR, DEFAULT 'active')] -> [balance (NUMERIC(15,2), DEFAULT 0.00)] -> [created_at (TIMESTAMP)]

2. TRANSACTIONS TABLE (Immutable Consolidated Financial Ledger Log)

[transaction_id (PK, SERIAL)] -> [sender_id (FK -> users.user_id, NULLABLE)] -> [receiver_id (FK -> users.user_id, NULLABLE)] -> [type (VARCHAR, CHECK IN ('deposit','withdraw','transfer'))] -> [amount (NUMERIC(15,2), CHECK > 0)] -> [description (TEXT)] -> [timestamp (TIMESTAMP)]

3. BENEFICIARIES TABLE (User-defined Peer Connectivity Lookup Intersection Graph)

[beneficiary_id (PK, SERIAL)] -> [user_id (FK -> users.user_id, CASCADE)] -> [beneficiary_user_id (FK -> users.user_id, CASCADE)] -> [nickname (VARCHAR)] -> [added_at (TIMESTAMP)] -> UNIQUE(user_id, beneficiary_user_id)

4. NOTIFICATIONS TABLE (System Generated Event Telemetry Audit Log)

[notification_id (PK, SERIAL)] -> [user_id (FK -> users.user_id, CASCADE)] -> [message (TEXT, NOT NULL)] -> [is_read (BOOLEAN, DEFAULT FALSE)] -> [created_at (TIMESTAMP)]

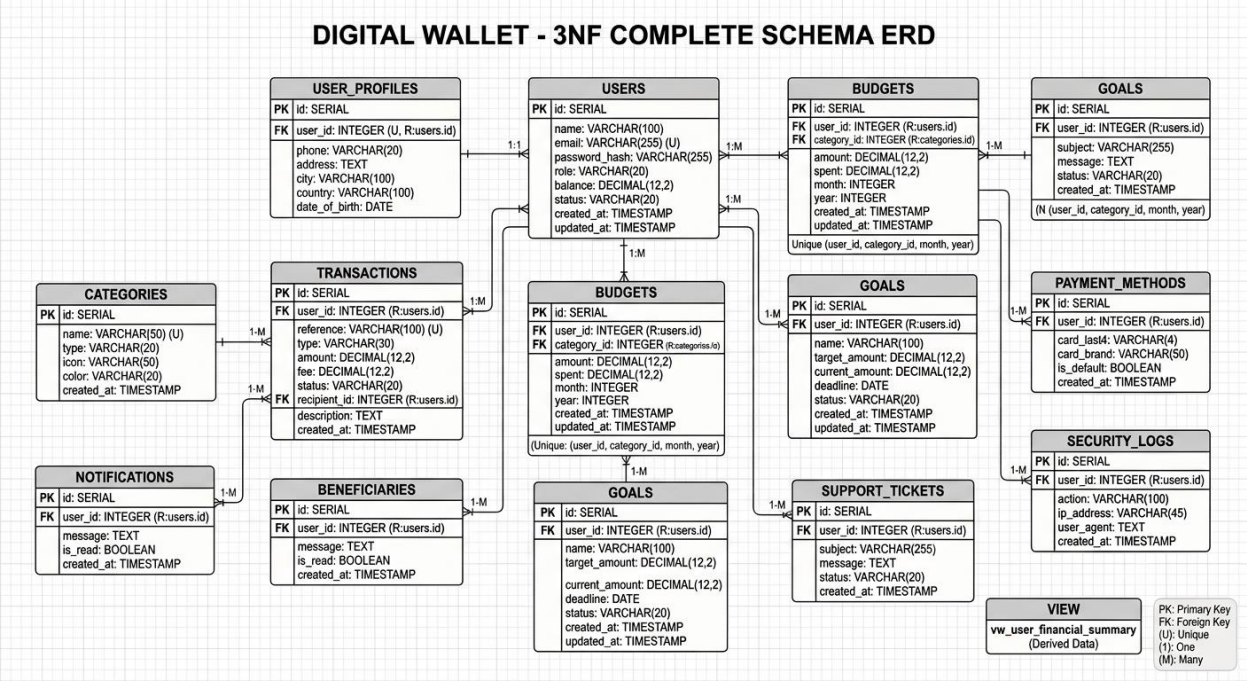

DIGITAL WALLET - 3NF COMPLETE SCHEMA ERD
USER_PROFILES
PK id: SERIAL
FK user_id: INTEGER (U, R:users.id)
phone: VARCHAR(20)
address: TEXT
city: VARCHAR(100)
country: VARCHAR(100)
date_of_birth: DATE
USERS
PK id: SERIAL
name: VARCHAR(100)
email: VARCHAR(255) (U)
password_hash: VARCHAR(255)
role: VARCHAR(20)
balance: DECIMAL(12,2)
status: VARCHAR(20)
created_at: TIMESTAMP
updated_at: TIMESTAMP
BUDGETS
PK id: SERIAL
FK user_id: INTEGER (R:users.id)
FK category_id: INTEGER (R:categories.id)
amount: DECIMAL(12,2)
spent: DECIMAL(12,2)
month: INTEGER
year: INTEGER
created_at: TIMESTAMP
updated_at: TIMESTAMP
Unique (user_id, category_id, month, year)
GOALS
PK id: SERIAL
FK user_id: INTEGER (R:users.id)
subject: VARCHAR(255)
message: TEXT
status: VARCHAR(20)
created_at: TIMESTAMP
(N (user_id, category_id, month, year)
CATEGORIES
PK id: SERIAL
name: VARCHAR(50) (U)
type: VARCHAR(20)
icon: VARCHAR(50)
color: VARCHAR(20)
created_at: TIMESTAMP
TRANSACTIONS
PK id: SERIAL
FK user_id: INTEGER (R:users.id)
reference: VARCHAR(100) (U)
type: VARCHAR(30)
amount: DECIMAL(12,2)
fee: DECIMAL(12,2)
status: VARCHAR(20)
FK recipient_id: INTEGER (R:users.id)
description: TEXT
created_at: TIMESTAMP
BUDGETS
PK id: SERIAL
FK user_id: INTEGER (R:users.id)
FK category_id: INTEGER (R:categoriss.id)
amount: DECIMAL(12,2)
spent: DECIMAL(12,2)
month: INTEGER
year: INTEGER
created_at: TIMESTAMP
updated_at: TIMESTAMP
(Unique: (user_id, category_id, month, year)
GOALS
PK id: SERIAL
FK user_id: INTEGER (R:users.id)
name: VARCHAR(100)
target_amount: DECIMAL(12,2)
current_amount: DECIMAL(12,2)
deadline: DATE
status: VARCHAR(20)
created_at: TIMESTAMP
updated_at: TIMESTAMP
PAYMENT_METHODS
PK id: SERIAL
FK user_id: INTEGER (R:users.id)
card_last4: VARCHAR(4)
card_brand: VARCHAR(50)
is_default: BOOLEAN
created_at: TIMESTAMP
NOTIFICATIONS
PK id: SERIAL
FK user_id: INTEGER (R:users.id)
message: TEXT
is_read: BOOLEAN
created_at: TIMESTAMP
BENEFICIARIES
PK id: SERIAL
FK user_id: INTEGER (R:users.id)
message: TEXT
is_read: BOOLEAN
created_at: TIMESTAMP
GOALS
PK id: SERIAL
FK user_id: INTEGER (R:users.id)
name: VARCHAR(100)
target_amount: DECIMAL(12,2)
current_amount: DECIMAL(12,2)
deadline: DATE
status: VARCHAR(20)
created_at: TIMESTAMP
updated_at: TIMESTAMP
SUPPORT_TICKETS
PK id: SERIAL
FK user_id: INTEGER (R:users.id)
subject: VARCHAR(255)
message: TEXT
status: VARCHAR(20)
created_at: TIMESTAMP
SECURITY_LOGS
PK id: SERIAL
FK user_id: INTEGER (R:users.id)
action: VARCHAR(100)
ip_address: VARCHAR(45)
user_agent: TEXT
created_at: TIMESTAMP
VIEW
vw_user_financial_summary
(Derived Data)
PK: Primary Key
FK: Foreign Key
(U): Unique
(1): One
(M): Many
1:1
1:M
1-M

Figure 3

Screenshots of Database Script

The following DDL initialization script constructs the active relational structures, indexes, and tables inside the serverless PostgreSQL instance:

```sql
-- ============================================
-- COMPLETE DIGITAL WALLET DATABASE SETUP
-- RUN THIS ENTIRE FILE IN NEON TECH SQL EDITOR
-- ============================================

-- ============================================
-- 1. USERS TABLE
-- ============================================
CREATE TABLE IF NOT EXISTS users (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    role VARCHAR(20) DEFAULT 'user'
        CHECK (role IN ('user', 'admin')),
    balance DECIMAL(12,2) DEFAULT 0.00
        CHECK (balance >= 0),
    status VARCHAR(20) DEFAULT 'active'
        CHECK (status IN ('active', 'blocked')),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```sql
-- ============================================
-- 17. SAMPLE COMPLEX QUERIES
-- ============================================

-- Query 1: User Financial Summary
SELECT '=== USER FINANCIAL SUMMARY ===' as info;
SELECT * FROM vw_user_financial_summary;

-- Query 2: Daily Activity
SELECT '=== DAILY ACTIVITY ===' as info;
SELECT * FROM vw_daily_activity LIMIT 10;

-- Query 3: Top Spenders
SELECT '=== TOP SPENDERS ===' as info;
SELECT
    u.name,
    COALESCE(SUM(t.amount), 0) as total_spent
FROM users u
LEFT JOIN transactions t ON u.id = t.user_id AND t.type IN ('sent', 'withdraw')
WHERE u.role != 'admin'
GROUP BY u.id, u.name
ORDER BY total_spent DESC
LIMIT 5;
```

```sql
-- DROP TABLE IF EXISTS goals CASCADE;
-- DROP TABLE IF EXISTS budgets CASCADE;
-- DROP TABLE IF EXISTS categories CASCADE;
-- DROP TABLE IF EXISTS notifications CASCADE;
-- DROP TABLE IF EXISTS beneficiaries CASCADE;
-- DROP TABLE IF EXISTS transactions CASCADE;
-- DROP TABLE IF EXISTS user_profiles CASCADE;
-- DROP TABLE IF EXISTS users CASCADE;
-- DROP VIEW IF EXISTS vw_user_financial_summary CASCADE;
-- DROP VIEW IF EXISTS vw_daily_activity CASCADE;
-- DROP FUNCTION IF EXISTS update_updated_at() CASCADE;

-- ============================================
-- 19. COMPLETE DATABASE INFO
-- ============================================
SELECT '=== DATABASE SETUP COMPLETE ===' as info;
SELECT 'Total Tables: 11' as info;
SELECT 'Total Views: 2' as info;
SELECT 'Total Triggers: 3' as info;
SELECT 'Sample Users: ' || COUNT(*) FROM users;
SELECT 'Sample Transactions: ' || COUNT(*) FROM transactions;
```

```sql
    CREATE INDEX idx_users_email ON users(email);
    CREATE INDEX idx_users_status ON users(status);
    CREATE INDEX idx_users_role ON users(role);

    -- ============================================
    -- 2. USER_PROFILES TABLE
    -- ============================================
    CREATE TABLE IF NOT EXISTS user_profiles (
        id SERIAL PRIMARY KEY,
        user_id INTEGER UNIQUE
            REFERENCES users(id) ON DELETE CASCADE,
        phone VARCHAR(20),
        address TEXT,
        city VARCHAR(100),
        country VARCHAR(100),
        date_of_birth DATE
    );

    CREATE INDEX idx_user_profiles_user_id ON user_profiles(user_id);

```

```sql
-- ============================================
-- 3. TRANSACTIONS TABLE
-- ============================================
CREATE TABLE IF NOT EXISTS transactions (
    id SERIAL PRIMARY KEY,
    reference VARCHAR(100) UNIQUE NOT NULL,
    user_id INTEGER
        REFERENCES users(id) ON DELETE CASCADE,
    type VARCHAR(30) NOT NULL
        CHECK (type IN ('deposit', 'withdraw', 'sent', 'received')),
    amount DECIMAL(12,2) NOT NULL
        CHECK (amount > 0),
    fee DECIMAL(12,2) DEFAULT 0
        CHECK (fee >= 0),
    status VARCHAR(20) DEFAULT 'completed'
        CHECK (status IN ('pending', 'completed', 'failed')),
    recipient_id INTEGER
        REFERENCES users(id) ON DELETE SET NULL,
    description TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_transactions_user_type ON transactions(user_id, type);
CREATE INDEX idx_transactions_created ON transactions(created_at DESC);
CREATE INDEX idx_transactions_user_id ON transactions(user_id);
```

```sql
CREATE INDEX idx_transactions_user_id ON transactions(user_id);
CREATE INDEX idx_transactions_reference ON transactions(reference);

-- ============================================
-- 4. BENEFICIARIES TABLE
-- ============================================
CREATE TABLE IF NOT EXISTS beneficiaries (
    id SERIAL PRIMARY KEY,
    user_id INTEGER
        REFERENCES users(id) ON DELETE CASCADE,
    beneficiary_name VARCHAR(100) NOT NULL,
    beneficiary_email VARCHAR(255),
    is_favorite BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT unique_user_beneficiary
        UNIQUE (user_id, beneficiary_email)
);

CREATE INDEX idx_beneficiaries_user_id ON beneficiaries(user_id);

--

-- 5. NOTIFICATIONS TABLE
-- ============================================
CREATE TABLE IF NOT EXISTS notifications (
    id SERIAL PRIMARY KEY,
    user_id INTEGER
        REFERENCES users(id) ON DELETE CASCADE,
    message TEXT NOT NULL,
    is_read BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_notifications_user_read ON notifications(user_id, is_read);
CREATE INDEX idx_notifications_user_id ON notifications(user_id);

-- ============================================
-- 6. CATEGORIES TABLE
-- ============================================
CREATE TABLE IF NOT EXISTS categories (
    id SERIAL PRIMARY KEY,
    name VARCHAR(50) UNIQUE NOT NULL,
    type VARCHAR(20) DEFAULT 'expense',
    icon VARCHAR(50),
    color VARCHAR(20),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```sql
-- 7. BUDGETS TABLE
-- ============================================
CREATE TABLE IF NOT EXISTS budgets (
    id SERIAL PRIMARY KEY,
    user_id INTEGER
        REFERENCES users(id) ON DELETE CASCADE,
    category_id INTEGER
        REFERENCES categories(id) ON DELETE CASCADE,
    amount DECIMAL(12,2) NOT NULL
        CHECK (amount > 0),
    spent DECIMAL(12,2) DEFAULT 0
        CHECK (spent >= 0),
    month INTEGER NOT NULL
        CHECK (month BETWEEN 1 AND 12),
    year INTEGER NOT NULL
        CHECK (year >= 2000),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(user_id, category_id, month, year)
);

CREATE INDEX idx_budgets_user_month ON budgets(user_id, month, year);

    -- ============================================
    -- 8. GOALS TABLE
    -- ============================================
    CREATE TABLE IF NOT EXISTS goals (
        id SERIAL PRIMARY KEY,
        user_id INTEGER
            REFERENCES users(id) ON DELETE CASCADE,
        name VARCHAR(100) NOT NULL,
        target_amount DECIMAL(12,2) NOT NULL
            CHECK (target_amount > 0),
        current_amount DECIMAL(12,2) DEFAULT 0
            CHECK (current_amount >= 0),
        deadline DATE,
        status VARCHAR(20) DEFAULT 'active'
            CHECK (status IN ('active', 'completed', 'cancelled')),
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    );

    CREATE INDEX idx_goals_user_status ON goals(user_id, status);
```

```sql
-- ============================================
-- 9. SUPPORT_TICKETS TABLE
-- ============================================
CREATE TABLE IF NOT EXISTS support_tickets (
    id SERIAL PRIMARY KEY,
    user_id INTEGER
        REFERENCES users(id) ON DELETE CASCADE,
    subject VARCHAR(255) NOT NULL,
    message TEXT NOT NULL,
    status VARCHAR(20) DEFAULT 'open'
        CHECK (status IN ('open', 'in_progress', 'resolved', 'closed')),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_support_tickets_status ON support_tickets(status);

-- ============================================
-- 10. PAYMENT_METHODS TABLE
-- ============================================
CREATE TABLE IF NOT EXISTS payment_methods (
    id SERIAL PRIMARY KEY,
    user_id INTEGER
        REFERENCES users(id) ON DELETE CASCADE,
    card_last4 VARCHAR(4) NOT NULL,
```

```sql
    card_last4 VARCHAR(4) NOT NULL,
    card_brand VARCHAR(50) NOT NULL,
    is_default BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- ============================================
-- 11. SECURITY_LOGS TABLE
-- ============================================
CREATE TABLE IF NOT EXISTS security_logs (
    id SERIAL PRIMARY KEY,
    user_id INTEGER
        REFERENCES users(id) ON DELETE SET NULL,
    action VARCHAR(100),
    ip_address VARCHAR(45),
    user_agent TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_security_logs_user_action ON security_logs(user_id, action);
```

```sql
-- 12. VIEWS
-- ========================================

-- View: User Financial Summary
CREATE OR REPLACE VIEW vw_user_financial_summary AS
SELECT
    u.id,
    u.name,
    u.email,
    u.balance,
    COALESCE(p.phone, 'Not Provided') as phone,
    COUNT(DISTINCT t.id) as transaction_count,
    COALESCE(SUM(CASE WHEN t.type IN ('sent', 'withdraw') THEN t.amount ELSE 0 END), 0) as total_outgoing,
    COALESCE(SUM(CASE WHEN t.type IN ('received', 'deposit') THEN t.amount ELSE 0 END), 0) as total_incoming,
    u.created_at as member_since,
    CASE
        WHEN u.balance = 0 THEN 'Zero Balance'
        WHEN u.balance < 100 THEN 'Low Balance'
        WHEN u.balance < 1000 THEN 'Medium Balance'
        ELSE 'High Balance'
    END as balance_category
FROM users u
LEFT JOIN user_profiles p ON u.id = p.user_id
LEFT JOIN transactions t ON u.id = t.user_id
WHERE u.role != 'admin'
```

```sql
        END as balance_category
    FROM users u
    LEFT JOIN user_profiles p ON u.id = p.user_id
    LEFT JOIN transactions t ON u.id = t.user_id
    WHERE u.role != 'admin'
    GROUP BY u.id, u.name, u.email, u.balance, p.phone, u.created_at;

    -- View: Daily Platform Activity
    CREATE OR REPLACE VIEW vw_daily_activity AS
    SELECT
        DATE(created_at) as date,
        COUNT(*) as total_transactions,
        SUM(amount) as total_volume,
        COUNT(DISTINCT user_id) as active_users,
        AVG(amount) as avg_transaction,
        SUM(CASE WHEN type = 'deposit' THEN amount ELSE 0 END) as deposits,
        SUM(CASE WHEN type = 'withdraw' THEN amount ELSE 0 END) as withdrawals,
        SUM(CASE WHEN type = 'sent' THEN amount ELSE 0 END) as sent,
        SUM(CASE WHEN type = 'received' THEN amount ELSE 0 END) as received
    FROM transactions
    GROUP BY DATE(created_at)
```

```sql
-- 13. TRIGGERS AND FUNCTIONS
-- ============================================

-- Function to update updated_at column
CREATE OR REPLACE FUNCTION update_updated_at()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Triggers for users table
CREATE TRIGGER trigger_users_updated_at
BEFORE UPDATE ON users
FOR EACH ROW
EXECUTE FUNCTION update_updated_at();

-- Triggers for budgets table
CREATE TRIGGER trigger_budgets_updated_at
BEFORE UPDATE ON budgets
FOR EACH ROW
EXECUTE FUNCTION update_updated_at();



-- Triggers for goals table
CREATE TRIGGER trigger_goals_updated_at
BEFORE UPDATE ON goals
FOR EACH ROW
EXECUTE FUNCTION update_updated_at();

-- ============================================
-- 14. SAMPLE DATA
-- ============================================

-- Insert Categories
INSERT INTO categories (name, type, icon, color) VALUES
('Food', 'expense', 'fa-utensils', '#48bb78'),
('Shopping', 'expense', 'fa-shopping-bag', '#f56565'),
('Bills', 'expense', 'fa-file-invoice-dollar', '#ed8936'),
('Entertainment', 'expense', 'fa-film', '#4299e1'),
('Transport', 'expense', 'fa-car', '#9f7aea'),
('Healthcare', 'expense', 'fa-heartbeat', '#e53e3e'),
('Education', 'expense', 'fa-graduation-cap', '#38b2ac'),
('Salary', 'income', 'fa-briefcase', '#48bb78'),
('Others', 'expense', 'fa-tag', '#a0aec0')
ON CONFLICT (name) DO NOTHING;

-- Insert Sample Users (password: password123)
INSERT INTO users (id, name, email, password_hash, role, balance, status) VALUES
```

```sql
-- Insert sample users (password: password123)
INSERT INTO users (id, name, email, password_hash, role, balance, status) VALUES
(1, 'Laiba Kanwal', 'bsse2480204@szabist.pk', '$2a$12$FDXyAsfaaegL4.MMuoubBeJpmcgaDV0l2RPwcvPHOXppv099bsoc2', 'user', 15000.00, 'active'),
(2, 'Shahla Abbasi', 'bsse2480221@szabist.pk', '$2a$12$FDXyAsfaaegL4.MMuoubBeJpmcgaDV0l2RPwcvPHOXppv099bsoc2', 'user', 8000.00, 'active'),
(3, 'Admin User', 'admin@digitalwallet.com', '$2a$12$FDXyAsfaaegL4.MMuoubBeJpmcgaDV0l2RPwcvPHOXppv099bsoc2', 'admin', 0.00, 'active')
ON CONFLICT (id) DO NOTHING;

-- Reset user ID sequence
SELECT setval('users_id_seq', (SELECT MAX(id) FROM users));

-- Insert User Profiles
INSERT INTO user_profiles (user_id, phone, address, city, country) VALUES
(1, '+92 300 1234567', '123 University Road', 'Karachi', 'Pakistan'),
(2, '+92 300 7654321', '456 College Street', 'Karachi', 'Pakistan')
ON CONFLICT (user_id) DO NOTHING;

-- Insert Sample Transactions
INSERT INTO transactions (reference, user_id, type, amount, description, status, created_at) VALUES
('DEP_JAN_001', 1, 'deposit', 5000.00, 'January Salary', 'completed', '2025-01-15 10:00:00'),
('DEP_FEB_001', 1, 'deposit', 5200.00, 'February Salary', 'completed', '2025-02-15 10:00:00'),
('DEP_MAR_001', 1, 'deposit', 5500.00, 'March Salary', 'completed', '2025-03-15 10:00:00'),
('DEP_APR_001', 1, 'deposit', 6000.00, 'April Salary', 'completed', '2025-04-15 10:00:00'),
('DEP_MAY_001', 1, 'deposit', 5800.00, 'May Salary', 'completed', '2025-05-15 10:00:00'),
('DEP_JUN_001', 1, 'deposit', 6200.00, 'June Salary', 'completed', '2025-06-15 10:00:00'),
('WTH_JAN_001', 1, 'withdraw', 2000.00, 'January Expenses', 'completed', '2025-01-20 14:30:00'),

-- Reset transaction sequence
SELECT setval('transactions_id_seq', (SELECT MAX(id) FROM transactions));

-- Insert Beneficiaries
INSERT INTO beneficiaries (user_id, beneficiary_name, beneficiary_email, is_favorite) VALUES
(1, 'Shahla Abbasi', 'bsse2480221@szabist.pk', true),
(2, 'Laiba Kanwal', 'bsse2480204@szabist.pk', true)
ON CONFLICT (user_id, beneficiary_email) DO NOTHING;

-- Insert Notifications
INSERT INTO notifications (user_id, message, is_read, created_at) VALUES
(1, 'Welcome to Digital Wallet!', true, NOW() - INTERVAL '30 days'),
(2, 'Welcome to Digital Wallet!', true, NOW() - INTERVAL '30 days'),
(1, '$500 deposited successfully!', false, NOW() - INTERVAL '5 days'),
(2, '$300 deposited successfully!', false, NOW() - INTERVAL '4 days');

-- Insert Budgets
INSERT INTO budgets (user_id, category_id, amount, month, year) VALUES
(1, (SELECT id FROM categories WHERE name = 'Food'), 500.00, EXTRACT(MONTH FROM NOW()), EXTRACT(YEAR FROM NOW())),
(1, (SELECT id FROM categories WHERE name = 'Shopping'), 300.00, EXTRACT(MONTH FROM NOW()), EXTRACT(YEAR FROM NOW())),
(1, (SELECT id FROM categories WHERE name = 'Bills'), 400.00, EXTRACT(MONTH FROM NOW()), EXTRACT(YEAR FROM NOW()))
ON CONFLICT (user_id, category_id, month, year) DO NOTHING;
```

```sql
-- Insert Goals
INSERT INTO goals (user_id, name, target_amount, current_amount, deadline) VALUES
(1, 'Vacation Fund', 1000.00, 200.00, '2026-12-31'),
(1, 'New Laptop', 800.00, 150.00, '2026-09-30'),
(2, 'Emergency Fund', 5000.00, 500.00, '2026-12-31');

-- Insert Support Tickets
INSERT INTO support_tickets (user_id, subject, message, status) VALUES
(1, 'Login Issue', 'I cannot login to my account after password reset', 'open'),
(2, 'Transaction Failed', 'My transaction failed but money was deducted', 'in_progress');

-- Insert Payment Methods
INSERT INTO payment_methods (user_id, card_last4, card_brand, is_default) VALUES
(1, '1234', 'visa', true),
(1, '5678', 'mastercard', false),
(2, '9012', 'visa', true);

-- Insert Security Logs
INSERT INTO security_logs (user_id, action, ip_address) VALUES
(1, 'login_success', '192.168.1.100'),
(1, 'login_success', '192.168.1.101'),
(2, 'login_success', '192.168.1.102'),
(1, 'password_change', '192.168.1.100'),
(2, 'login_failed', '192.168.1.200');

```

Screenshots of API Core Routes

The ExpressJS backend architecture controls incoming transactions, user states, and authentication processes via the following core controllers:

```
1   const express = require('express');
2   const cors = require('cors');
3   const path = require('path');
4   require('dotenv').config();
5
6   const { testConnection } = require('./db/database');
7   const userRoutes = require('./routes/users');
8   const transactionRoutes = require('./routes/transactions');
9   const adminRoutes = require('./routes/admin');
10
11  const app = express();
12  const PORT = process.env.PORT || 5000;
13
14  // Middleware
15  app.use(cors());
```

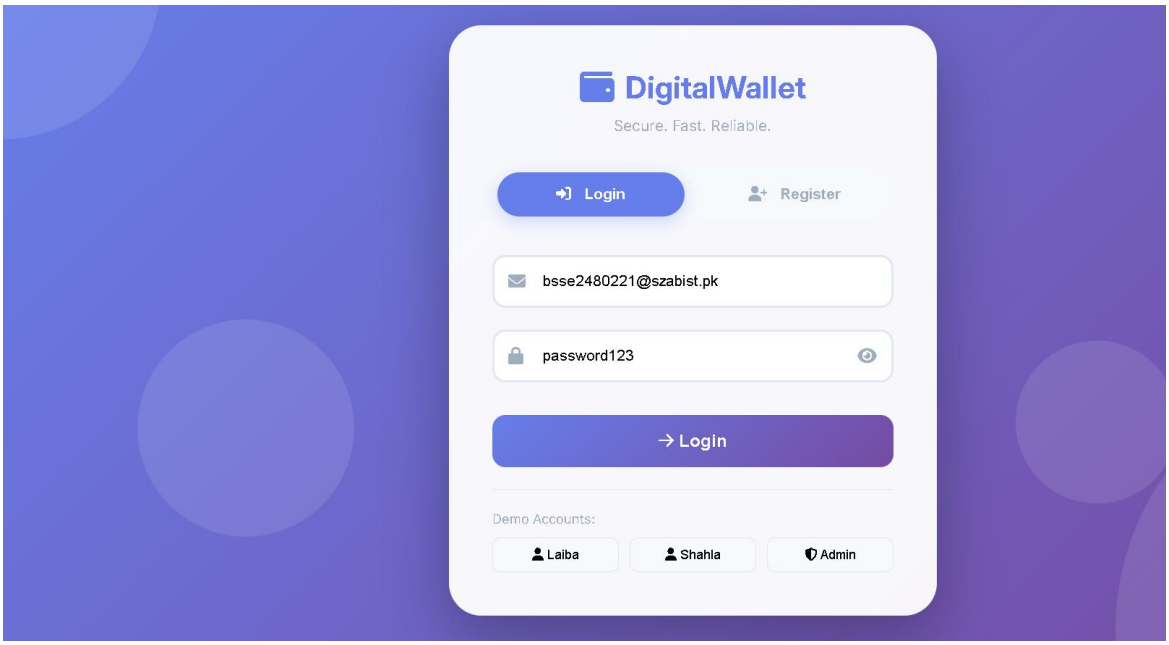
```
15  app.use(cors());
16  app.use(express.json());
17  app.use(express.urlencoded({ extended: true }));
18
19  // Serve static files from public directory
20  app.use(express.static(path.join(__dirname, '../public')));
21
22  // API Routes
23  app.use('/api/users', userRoutes);
24  app.use('/api/transactions', transactionRoutes);
25  app.use('/api/admin', adminRoutes);
26
27  // Health check endpoint
28  app.get('/api/health', (req, res) => {
29      res.json({
```

```
app.get('/api/health', (req, res) => {
    res.json({
        status: 'OK',
        message: 'Digital Wallet API is running',
        timestamp: new Date().toISOString()
    });
});

// Catch-all route for frontend
app.get('*', (req, res) => {
    res.sendFile(path.join(__dirname, '../public/index.html'));
});

// Error handling middleware
app.use((err, req, res, next) => {
    console.error('Error:', err);
    res.status(500).json({ error: 'Something went wrong!' });
});

// Start server
const startServer = async () => {
    console.log('🔄 Starting Digital Wallet Server...');
    console.log('🔍 Checking database connection...');

    const dbConnected = await testConnection();
    if (!dbConnected) {
        console.error('❌ Cannot start server without database');
        process.exit(1);
    }

    app.listen(PORT, '0.0.0.0', () => {
        console.log(`✅ Server running on http://localhost:${PORT}`);
        console.log(`✅ Open your browser at the Codespace URL on port ${PORT}`);
        console.log(`\n📋 Test Accounts:`);
        console.log(`   Laiba: bsse2480204@szabist.pk / password123`);
        console.log(`   Shahla: bsse2480221@szabist.pk / password123`);
        console.log(`   Admin: admin@digitalwallet.com / password123`);
    });
};

startServer();
```

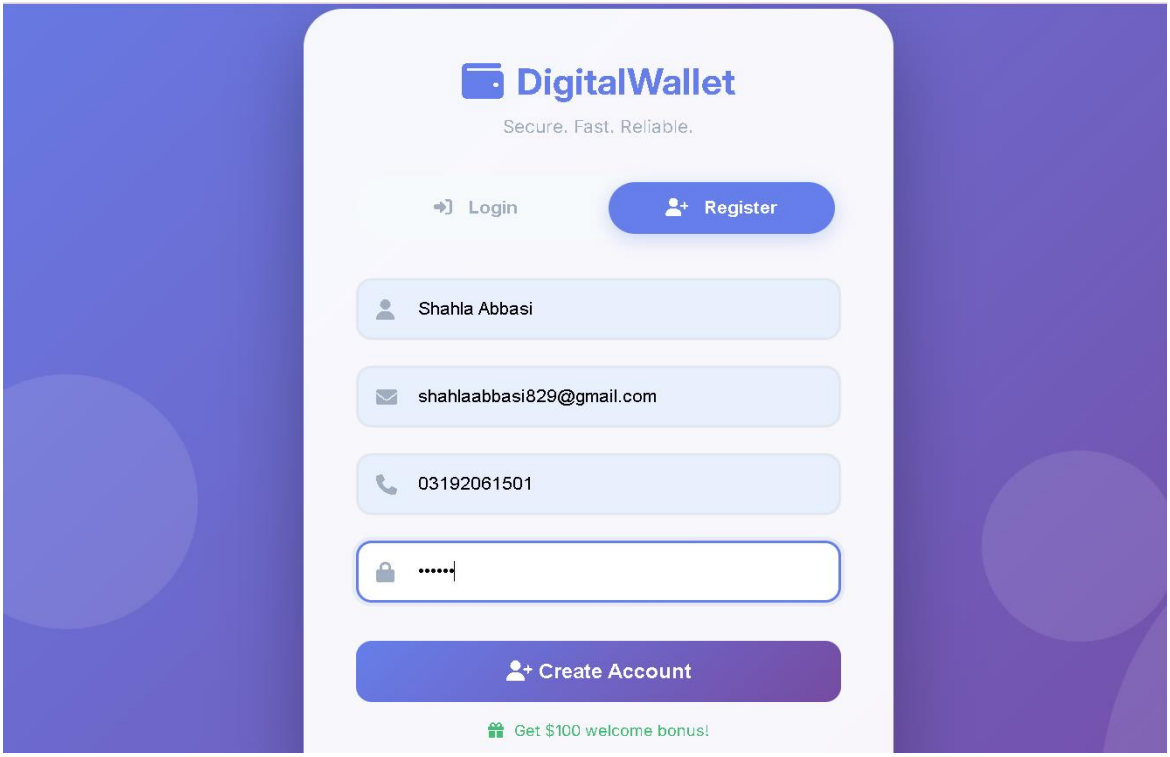
Screenshots of Client Frontend Dashboard

The client interfaces render ledger metrics, administrative switches, and responsive layouts across different viewport sizes:

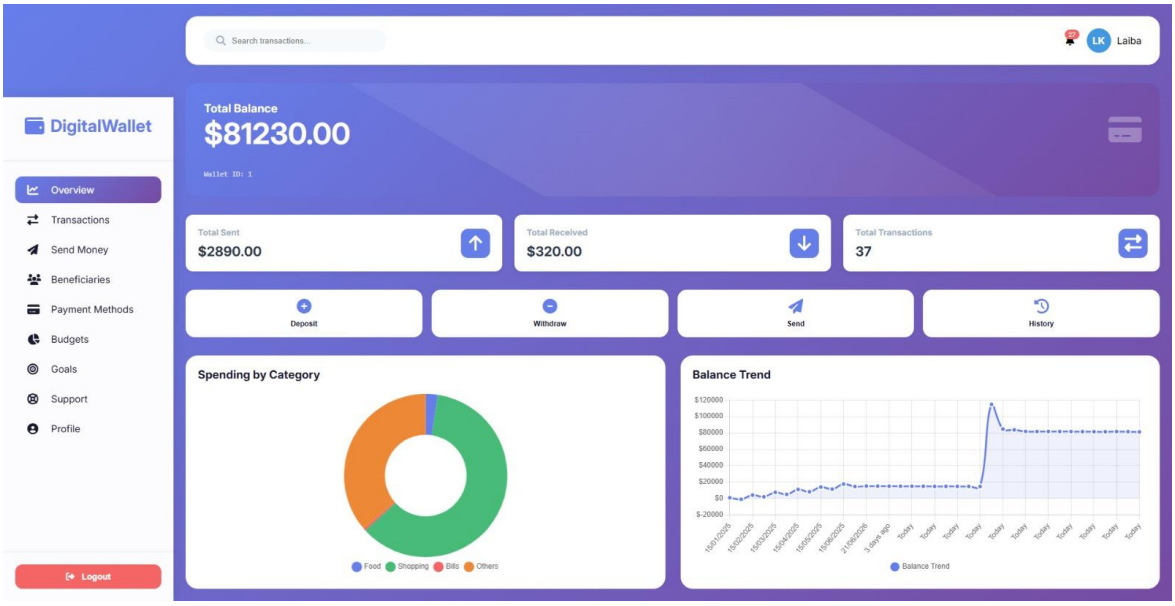
LOGIN:



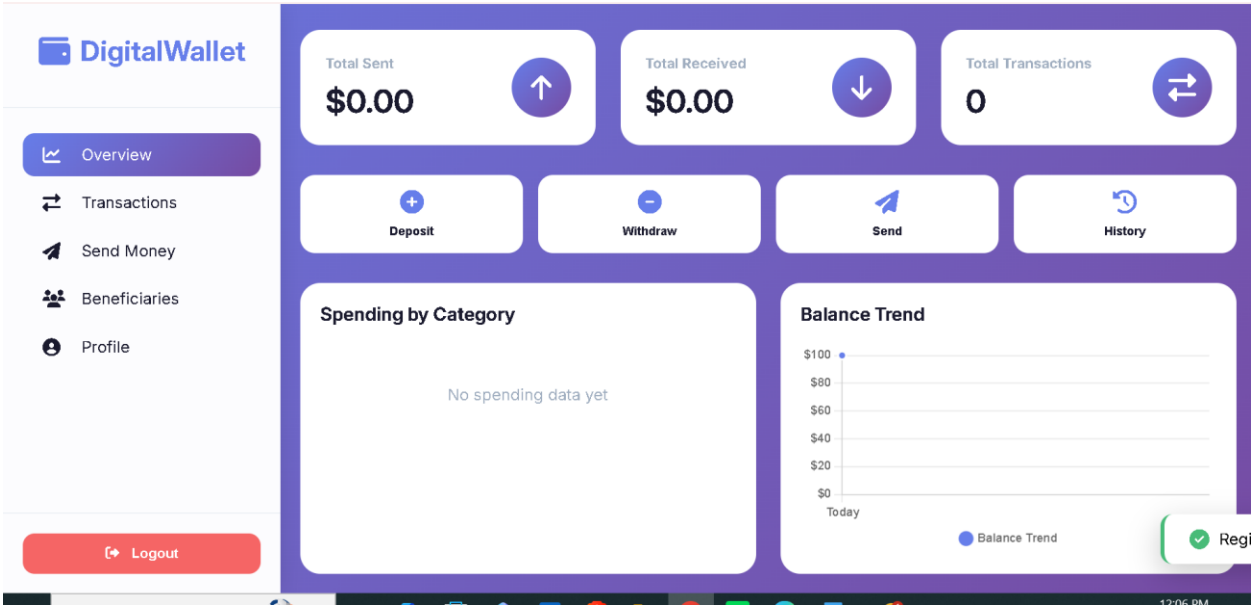
REGISTER:



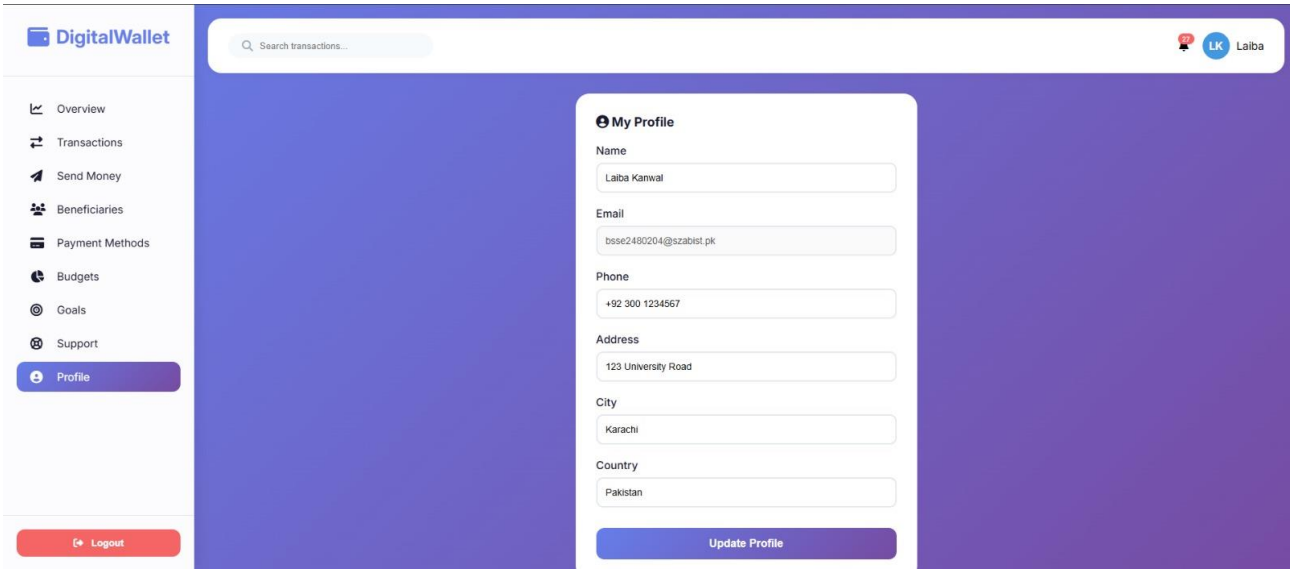
DASHBOARD:



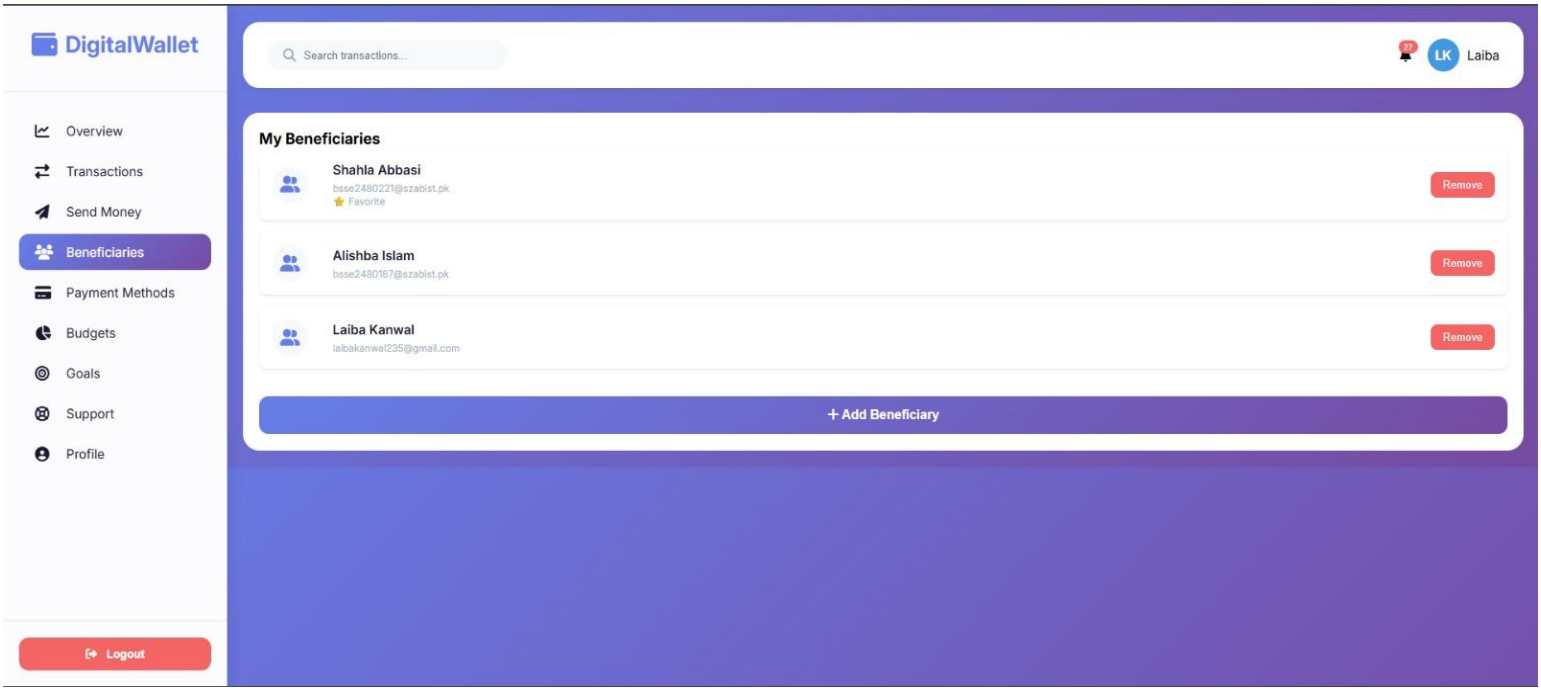
OVERVIEW BEFORE ANY TRANSCTIONS:

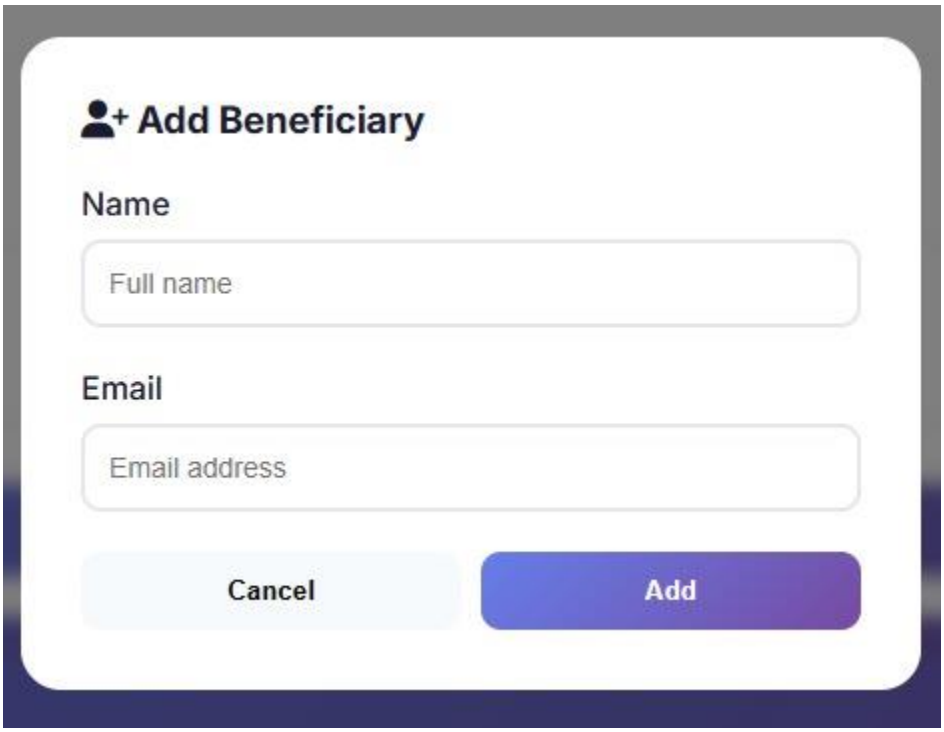


PROFILE:

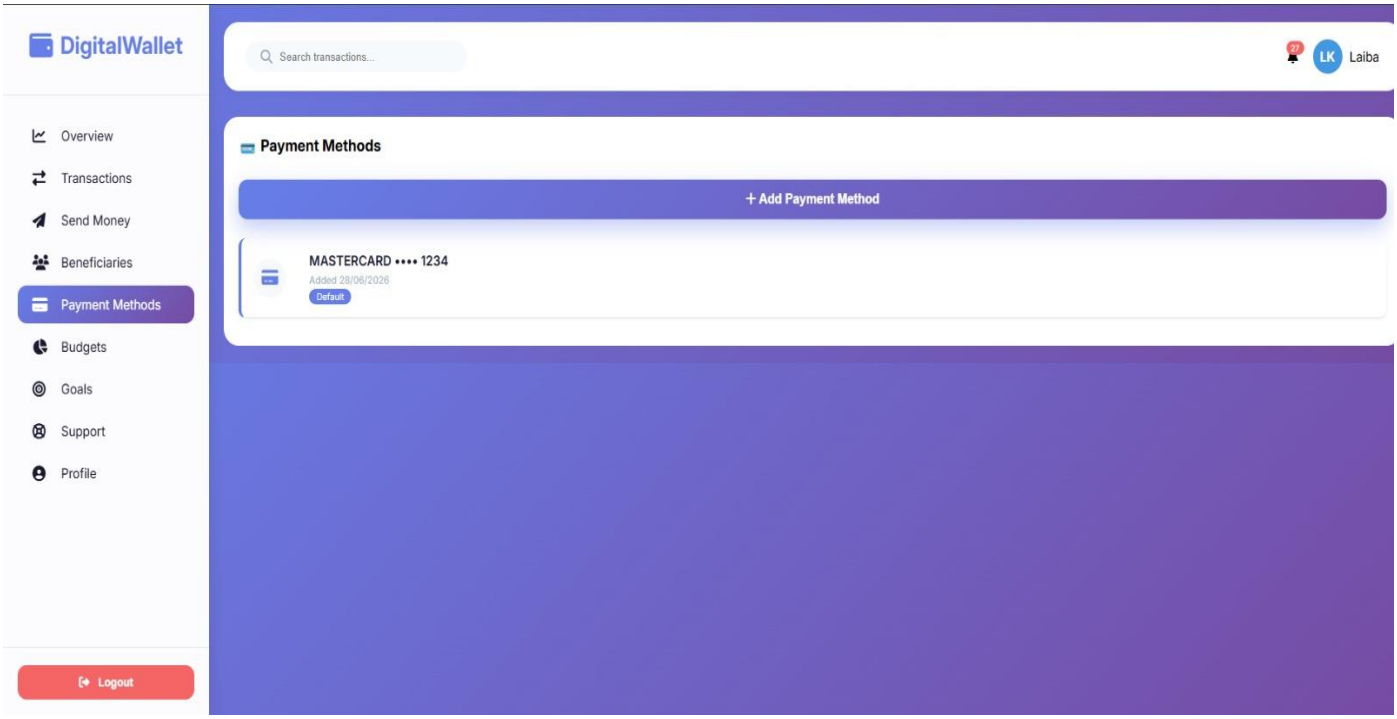


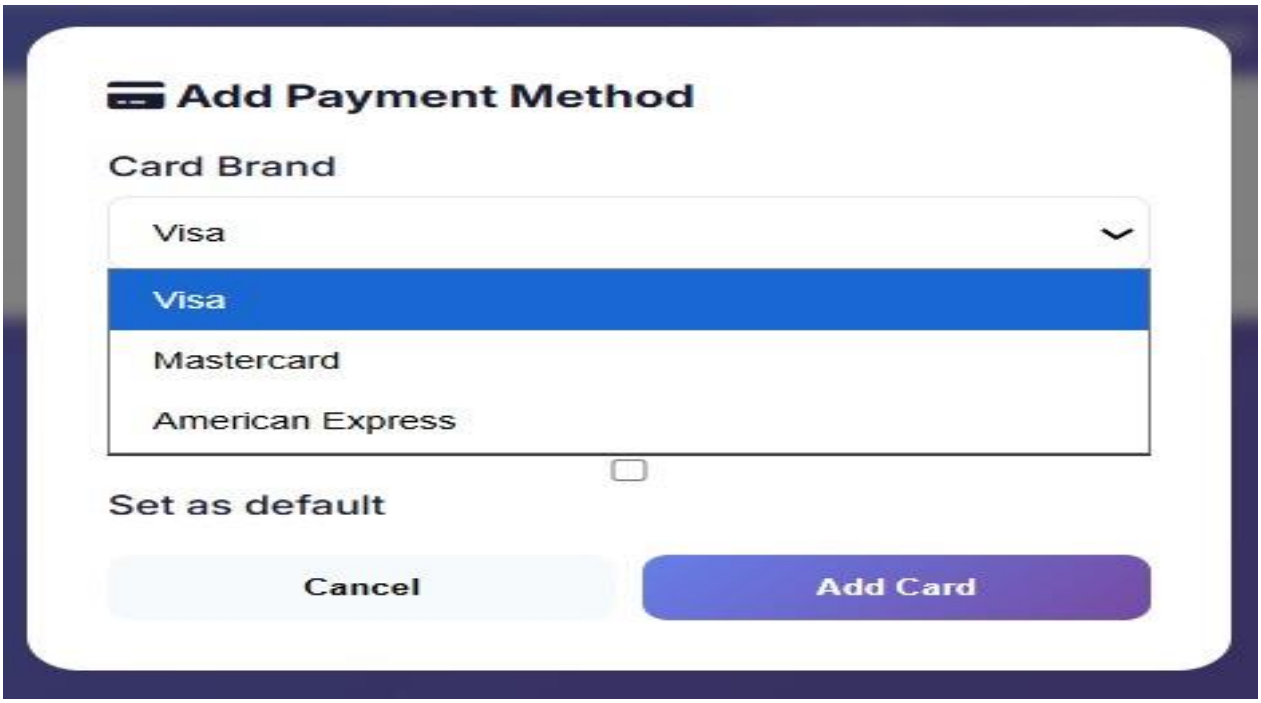
BENEFICIARIES:



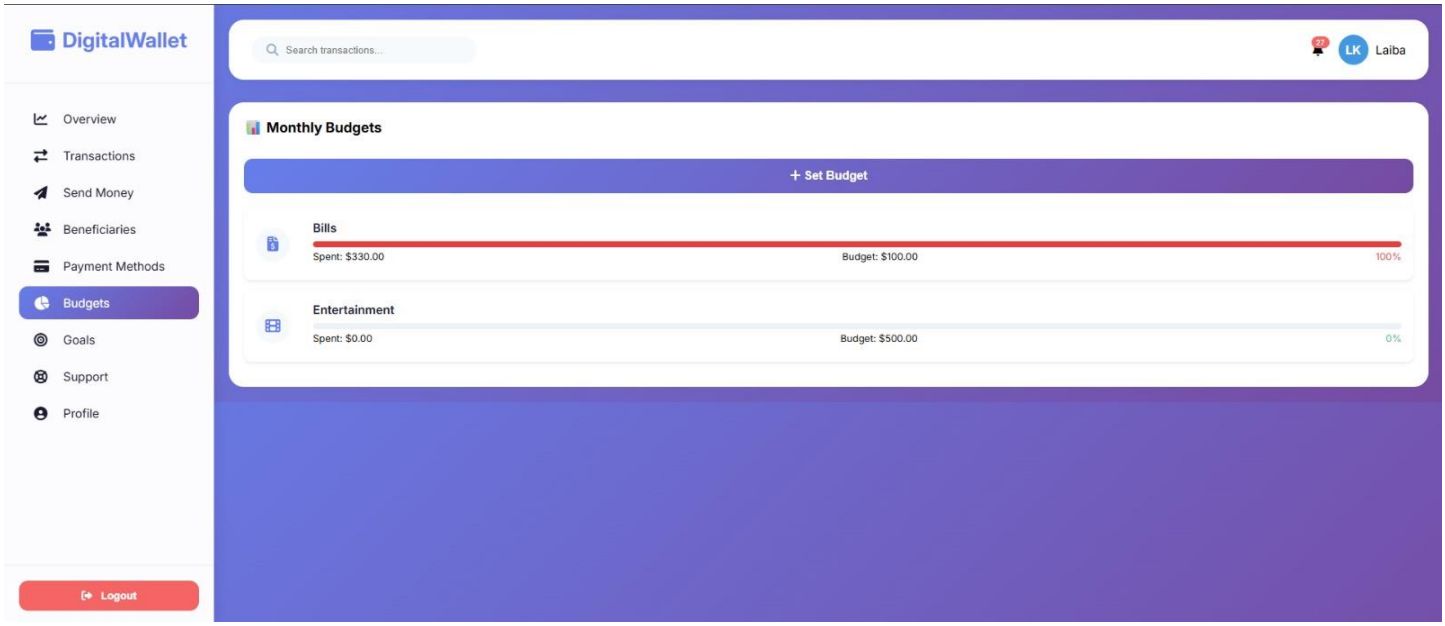


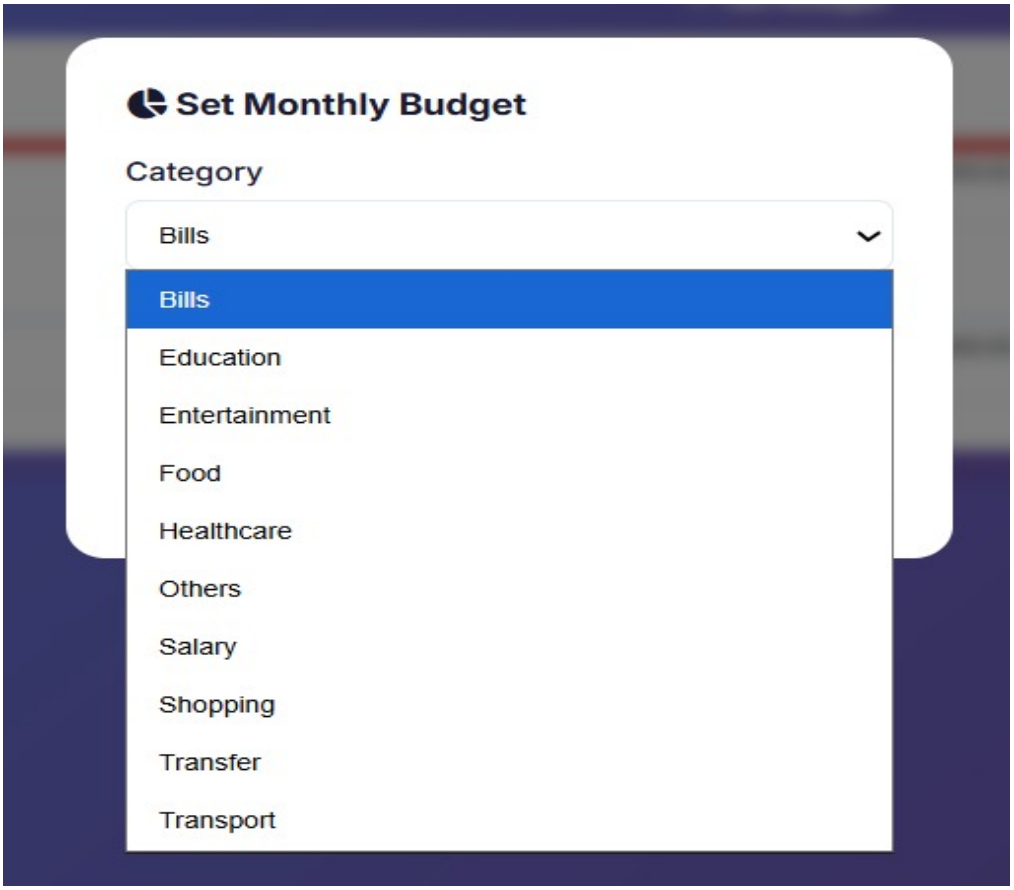
PAYMENT METHOD:

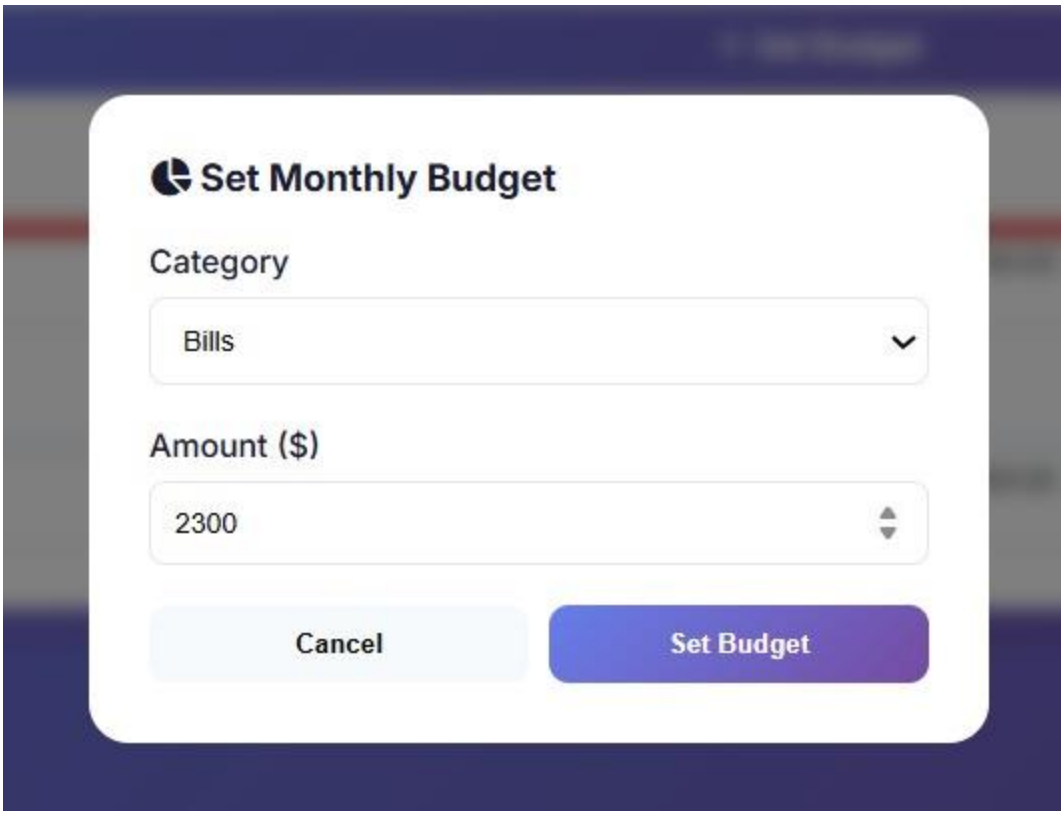




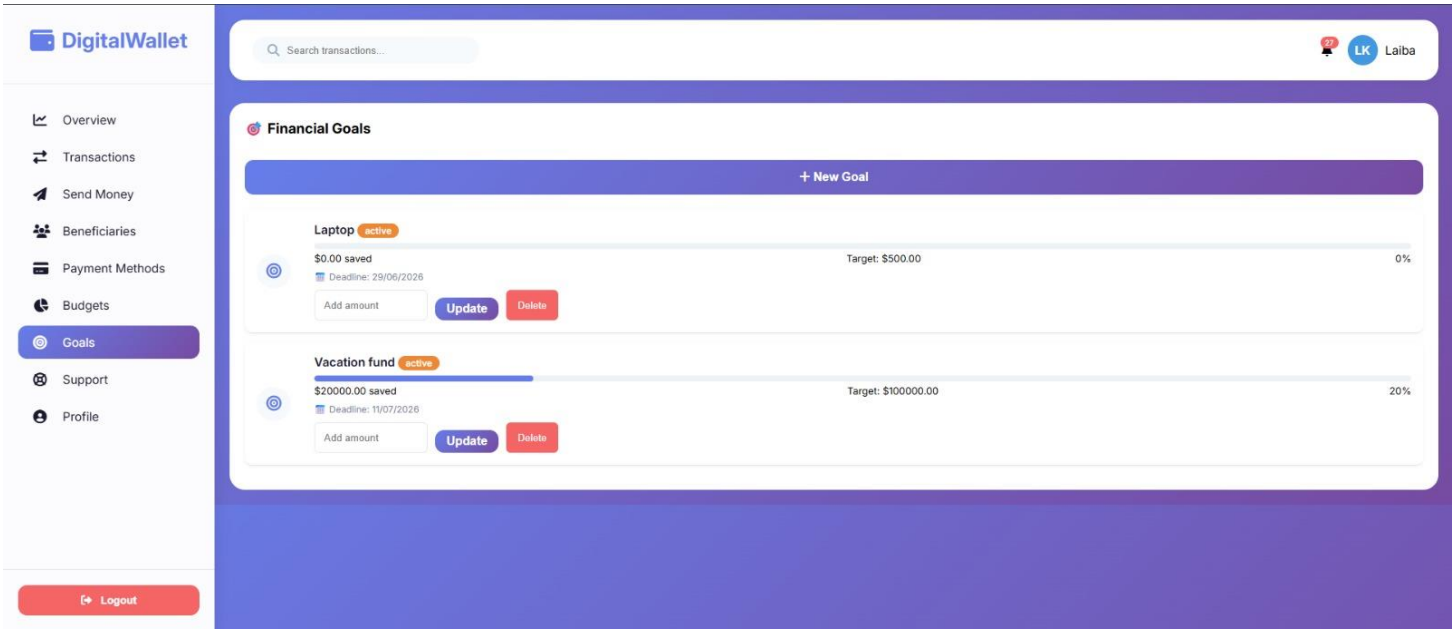
Budget:

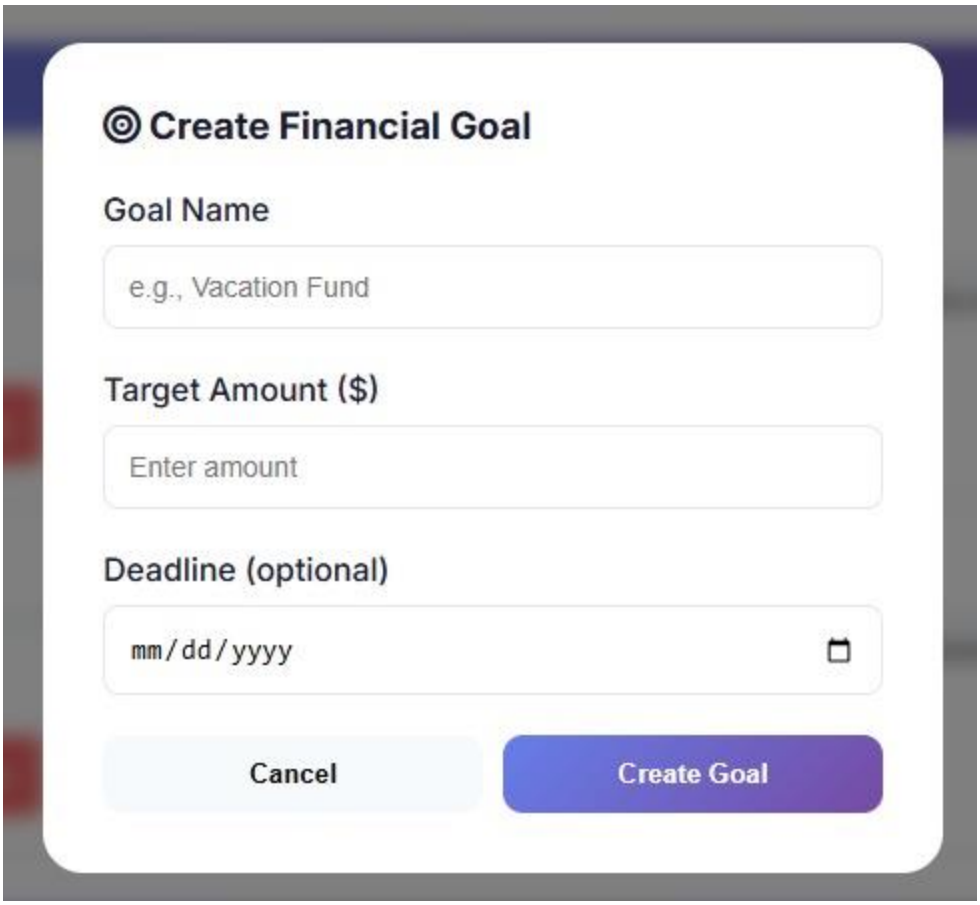

DigitalWallet
Search transactions...
27
LK
Laiba
Overview
Transactions
Send Money
Beneficiaries
Payment Methods
Budgets
Goals
Support
Profile
Logout
Monthly Budgets
+ Set Budget
Bills
Spent: $330.00
Budget: $100.00
100%
Entertainment
Spent: $0.00
Budget: $500.00
0%

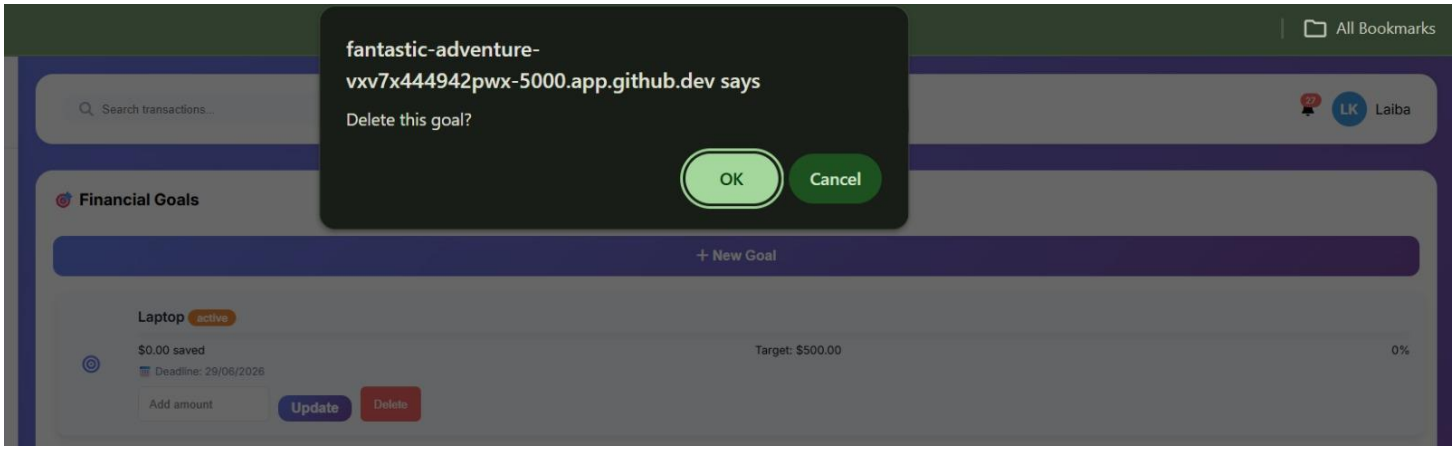

Set Monthly Budget
Category
Bills
Bills
Education
Entertainment
Food
Healthcare
Others
Salary
Shopping
Transfer
Transport


Set Monthly Budget
Category
Bills
Amount ($)
2300
Cancel
Set Budget

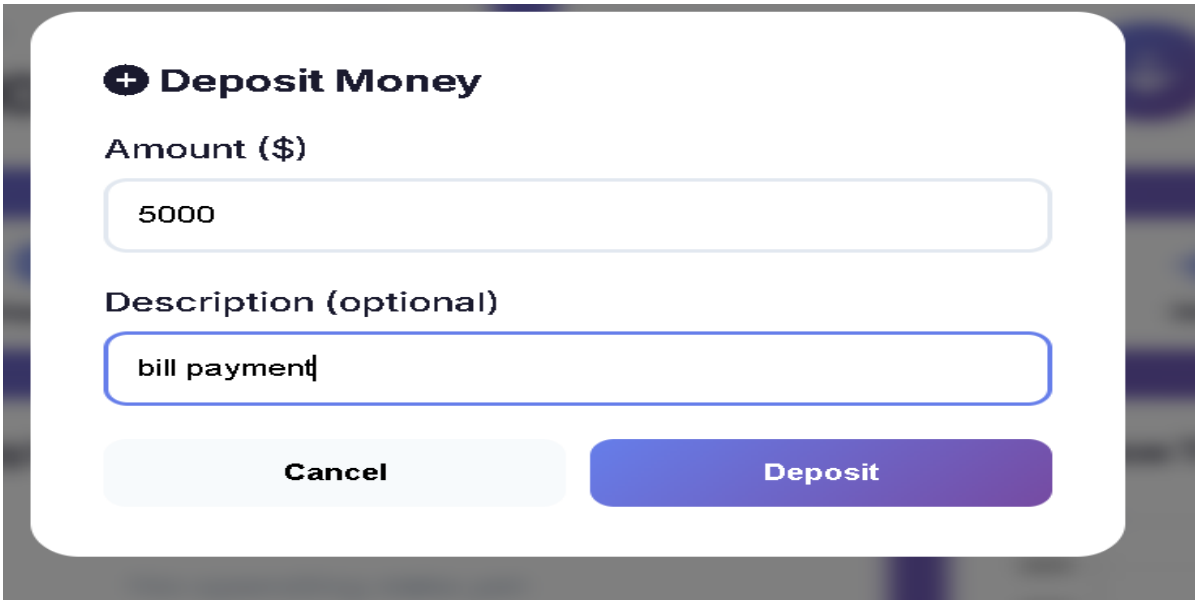
Goals:


DigitalWallet
Search transactions...
LK
Laiba
Overview
Transactions
Send Money
Beneficiaries
Payment Methods
Budgets
Goals
Support
Profile
Logout
Financial Goals
+ New Goal
Laptop
active
$0.00 saved
Target: $500.00
0%
Deadline: 29/06/2026
Add amount
Update
Delete
Vacation fund
active
$20000.00 saved
Target: $100000.00
20%
Deadline: 11/07/2026
Add amount
Update
Delete

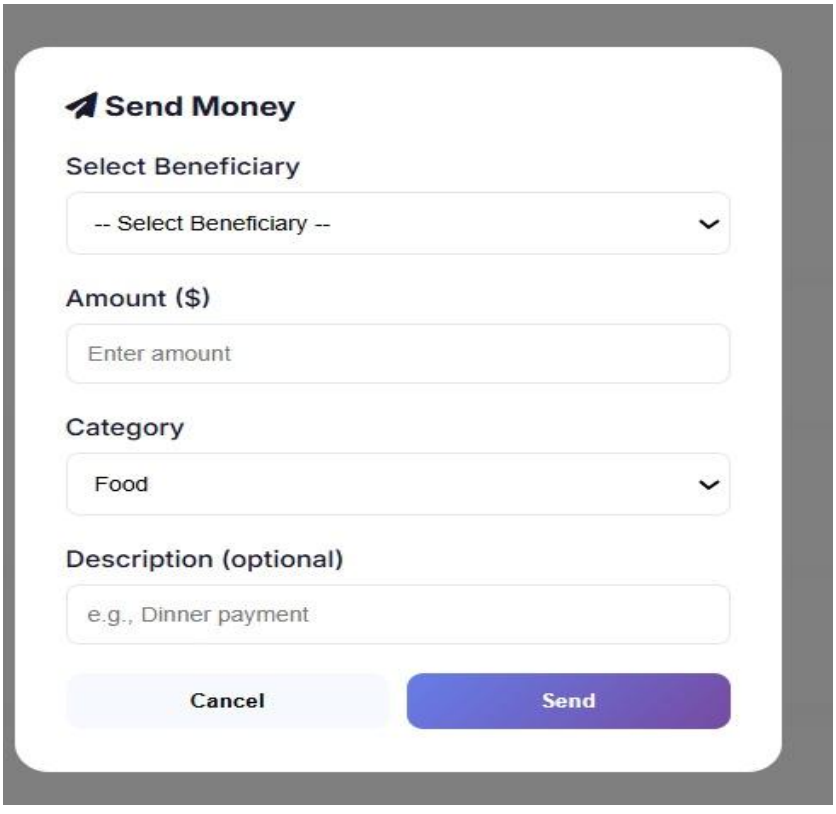

Create Financial Goal
Goal Name
e.g., Vacation Fund
Target Amount ($)
Enter amount
Deadline (optional)
mm/dd/yyyy
Cancel
Create Goal

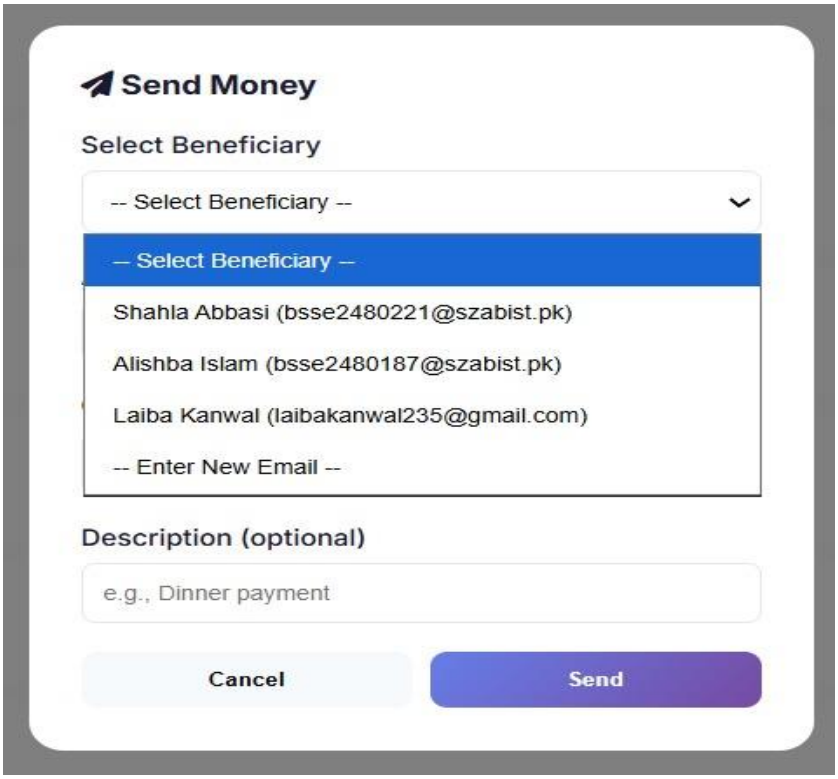

All Bookmarks
fantastic-adventure-vxv7x444942pwx-5000.app.github.dev says
Delete this goal?
OK
Cancel
Search transactions...
Laiba
Financial Goals
+ New Goal
Laptop
active
$0.00 saved
Target: $500.00
0%
Deadline: 29/06/2026
Add amount
Update
Delete

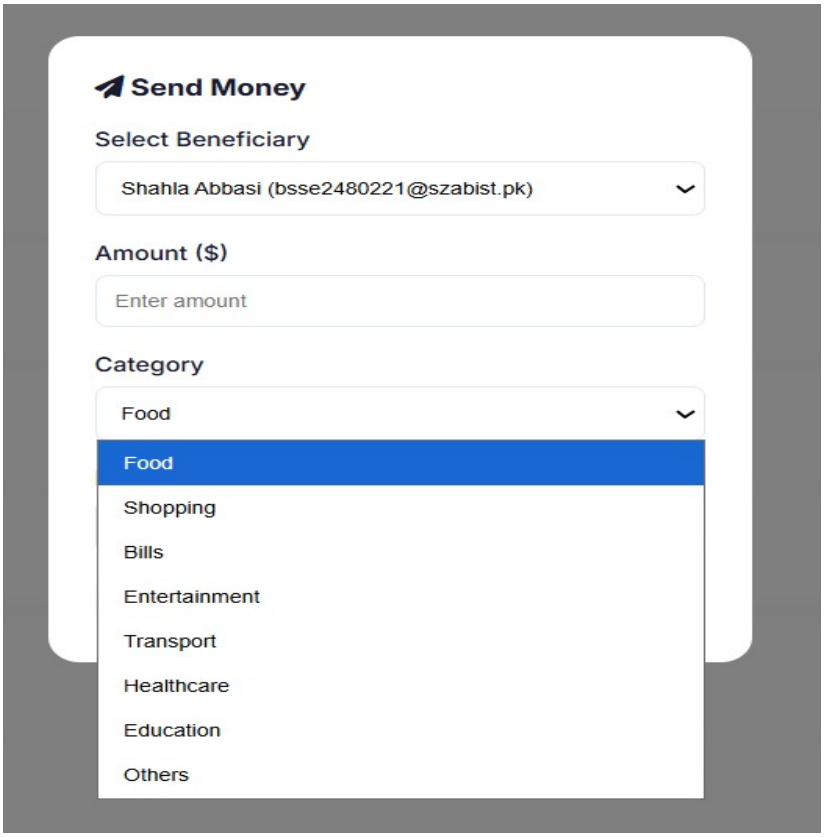
DEPOSIT MONEY:



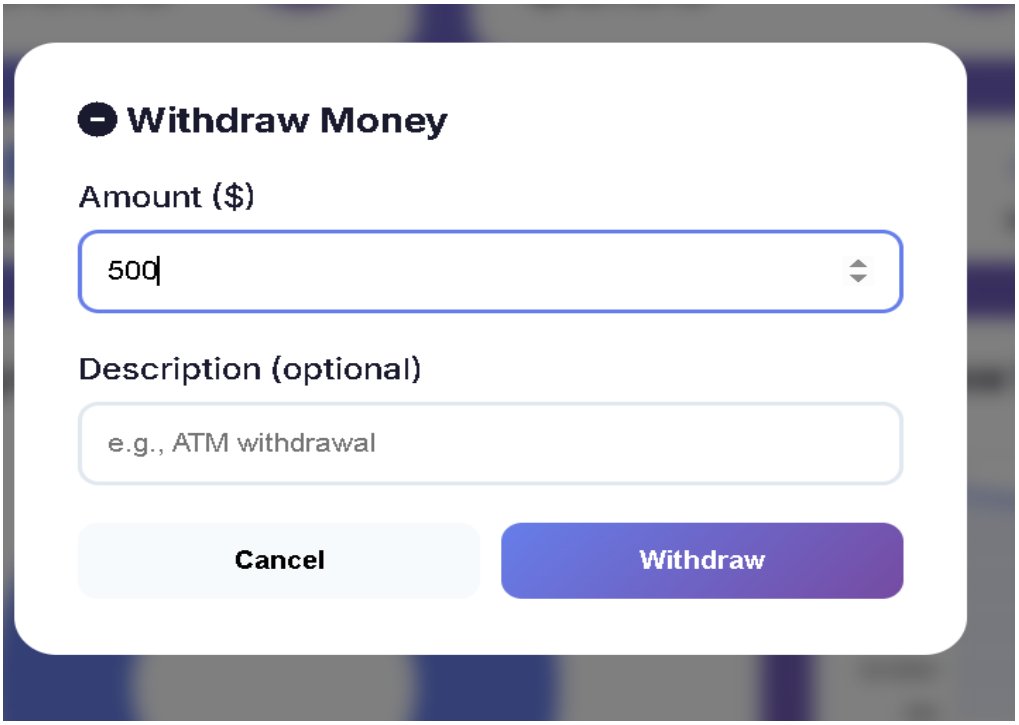
SENDING MONEY:



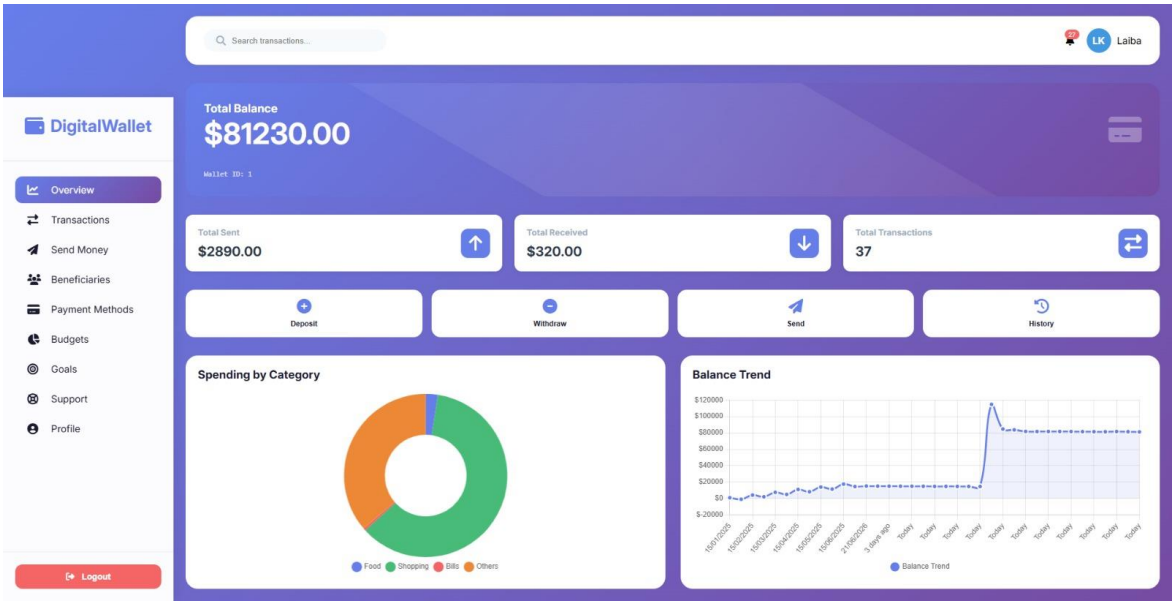

Send Money
Select Beneficiary
-- Select Beneficiary --
-- Select Beneficiary --
Shahla Abbasi (bsse2480221@szabist.pk)
Alishba Islam (bsse2480187@szabist.pk)
Laiba Kanwal (laibakanwal235@gmail.com)
-- Enter New Email --
Description (optional)
e.g., Dinner payment
Cancel
Send


Send Money
Select Beneficiary
Shahla Abbasi (bsse2480221@szabist.pk)
Amount ($)
Enter amount
Category
Food
Food
Shopping
Bills
Entertainment
Transport
Healthcare
Education
Others

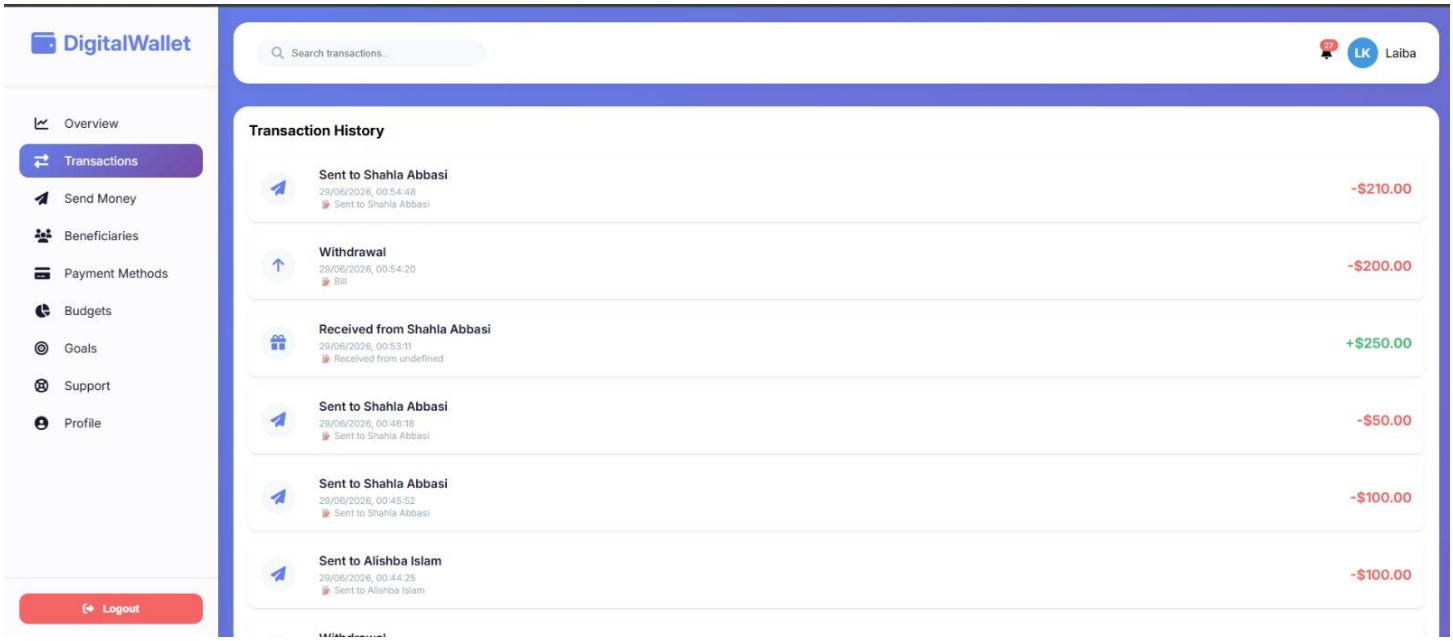
WITHDRAW MONEY:



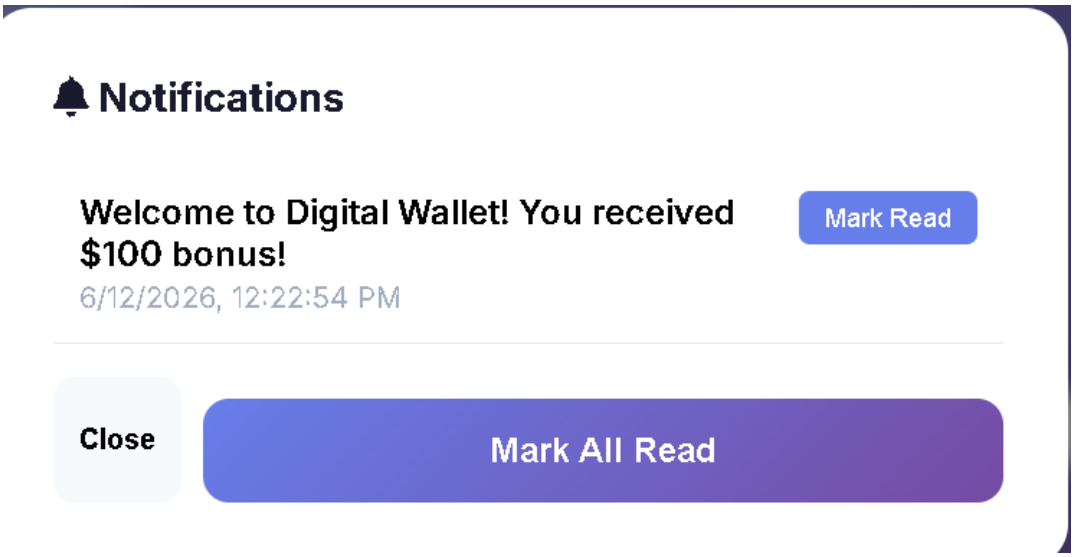
DASHBOARD AFTER TRANCTIONS:



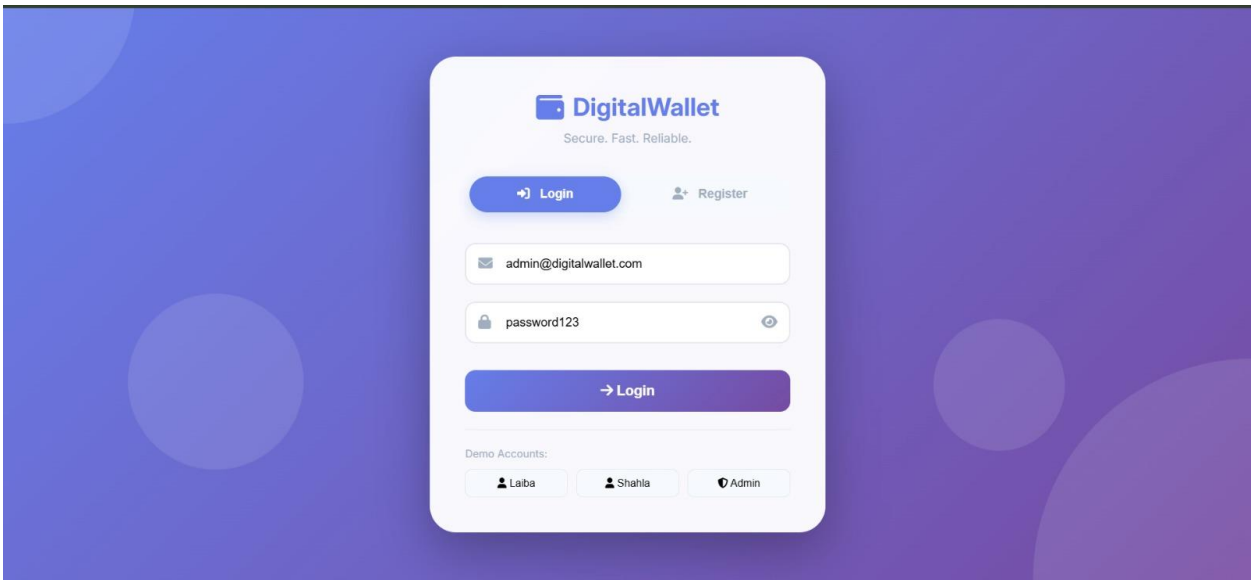
HISTORY:

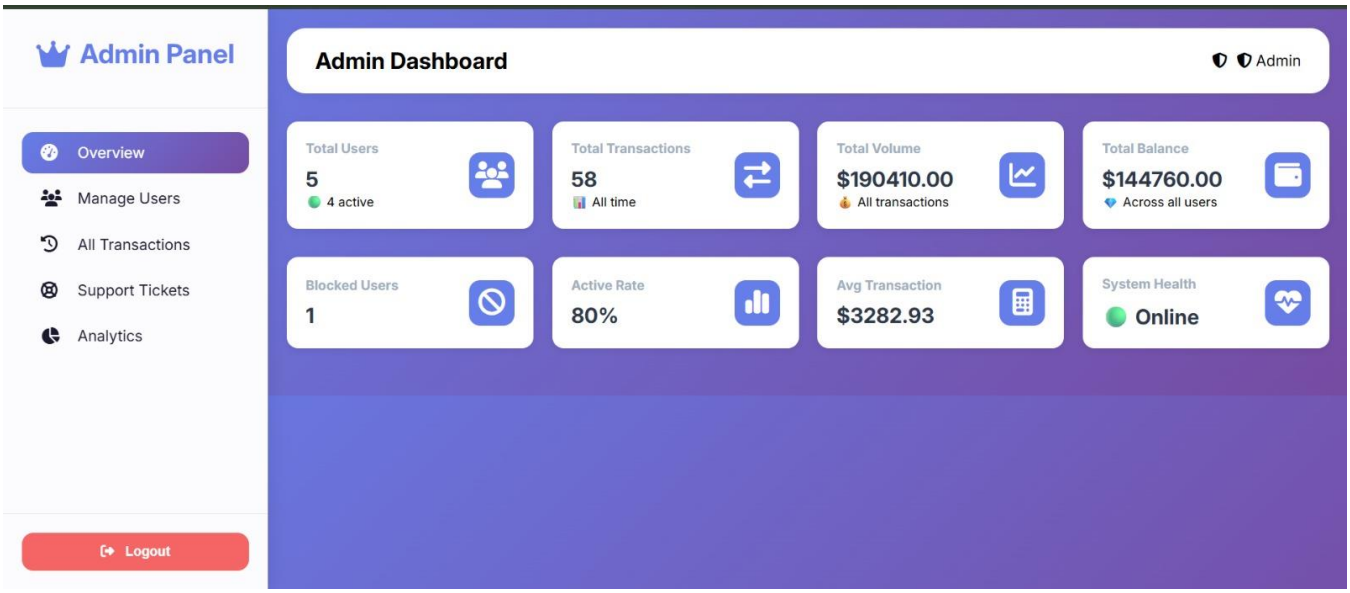


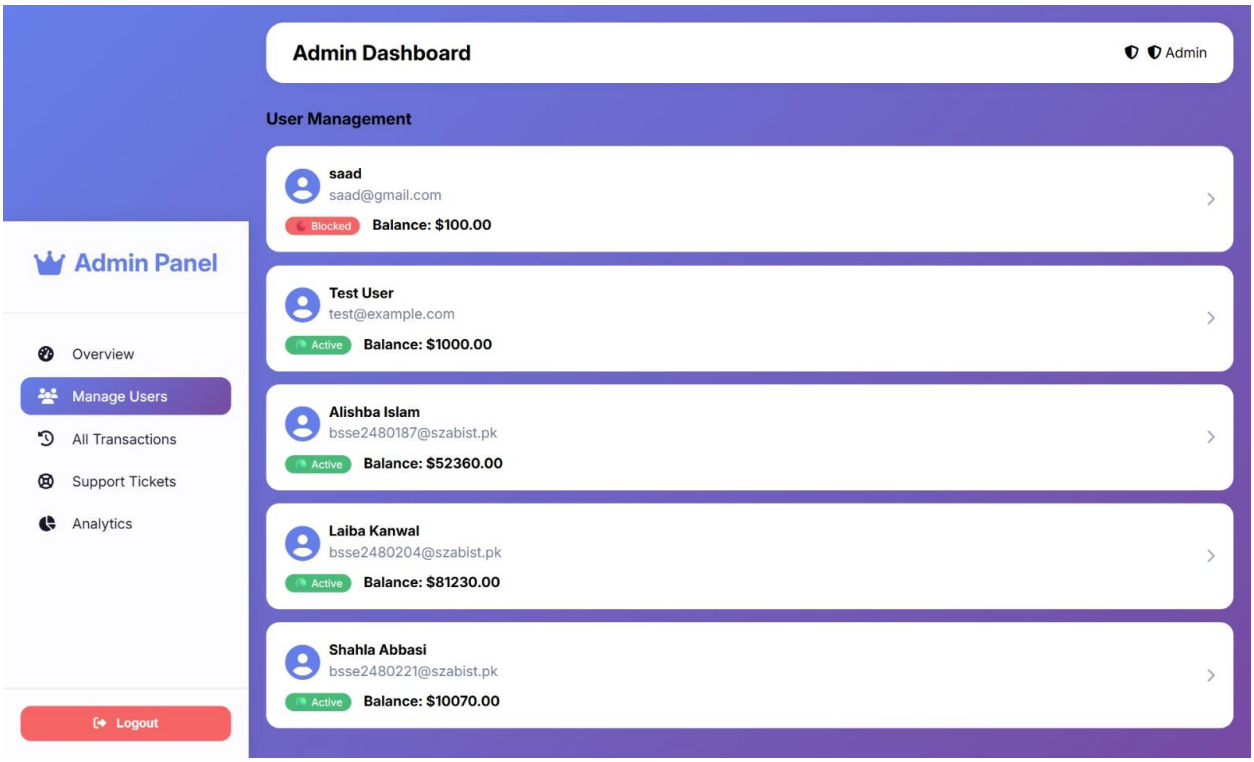
NOTIFICATION:

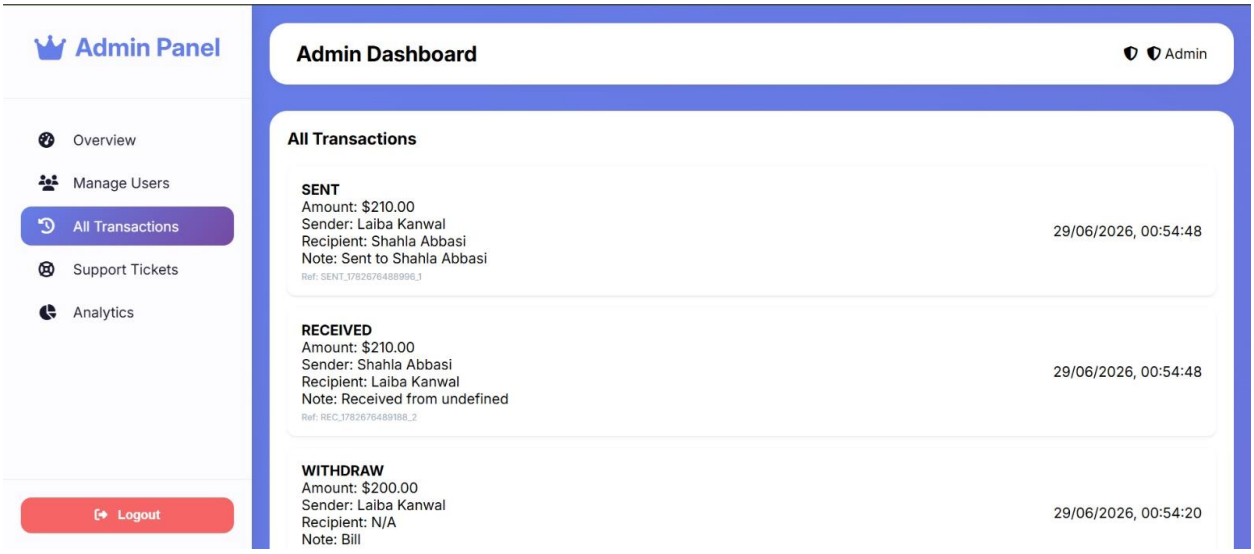


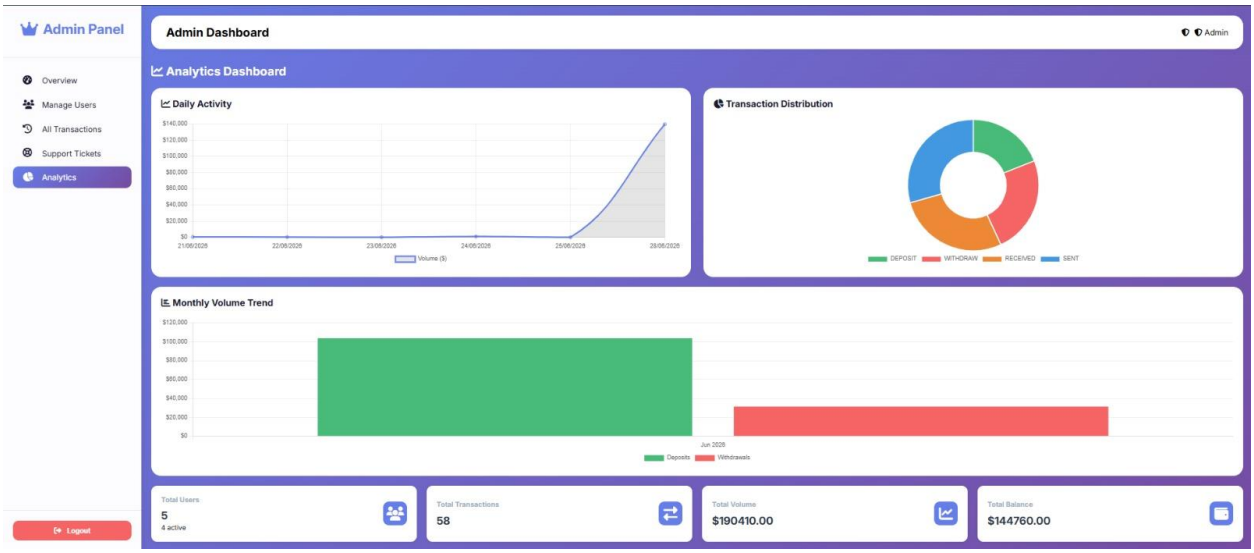
ADMIN PANEL:

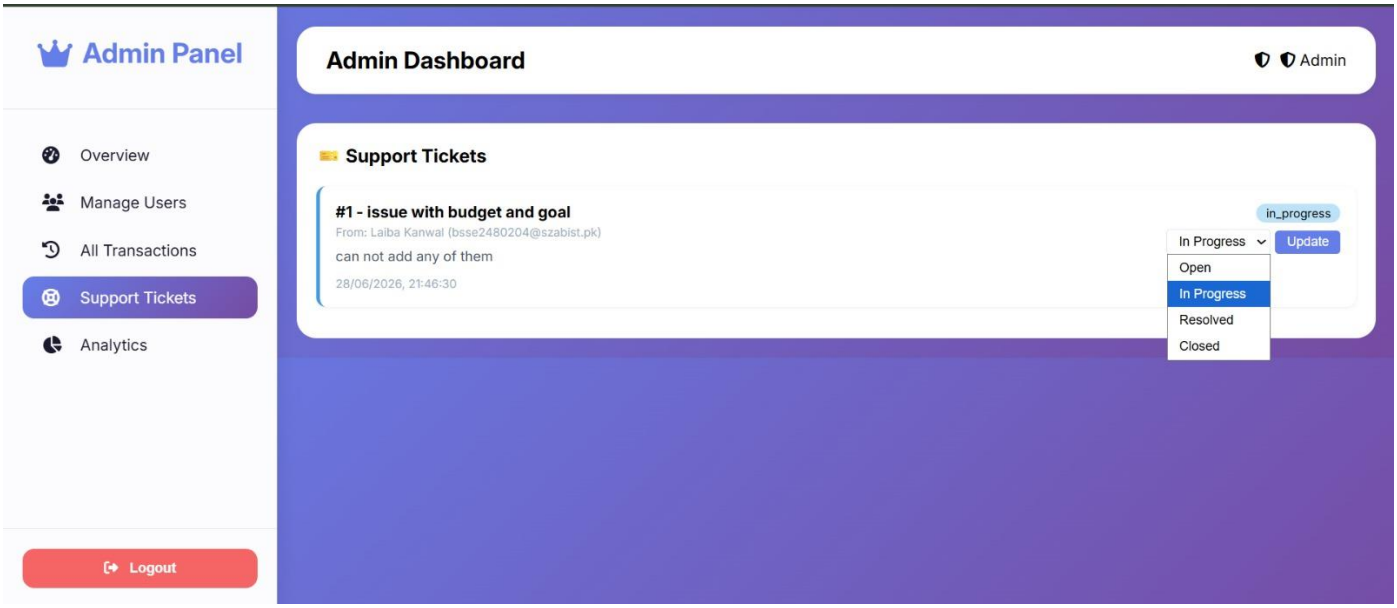

DigitalWallet
Secure. Fast. Reliable.
Login
Register
admin@digitalwallet.com
password123
Login
Demo Accounts:
Laiba
Shahla
Admin


Admin Panel
Overview
Manage Users
All Transactions
Support Tickets
Analytics
Logout
Admin Dashboard
Admin
Total Users
5
4 active
Total Transactions
58
All time
Total Volume
$190410.00
All transactions
Total Balance
$144760.00
Across all users
Blocked Users
1
Active Rate
80%
Avg Transaction
$3282.93
System Health
Online


Admin Dashboard
Admin
User Management
saad
saad@gmail.com
Blocked
Balance: $100.00
Test User
test@example.com
Active
Balance: $1000.00
Alishba Islam
bsse2480187@szabist.pk
Active
Balance: $52360.00
Laiba Kanwal
bsse2480204@szabist.pk
Active
Balance: $81230.00
Shahla Abbasi
bsse2480221@szabist.pk
Active
Balance: $10070.00
Admin Panel
Overview
Manage Users
All Transactions
Support Tickets
Analytics
Logout


Admin Panel
Overview
Manage Users
All Transactions
Support Tickets
Analytics
Logout
Admin Dashboard
Admin
All Transactions
SENT
Amount: $210.00
Sender: Laiba Kanwal
Recipient: Shahla Abbasi
Note: Sent to Shahla Abbasi
29/06/2026, 00:54:48
RECEIVED
Amount: $210.00
Sender: Shahla Abbasi
Recipient: Laiba Kanwal
Note: Received from undefined
29/06/2026, 00:54:48
WITHDRAW
Amount: $200.00
Sender: Laiba Kanwal
Recipient: N/A
Note: Bill
29/06/2026, 00:54:20


Admin Panel
Admin Dashboard
Admin
Overview
Manage Users
All Transactions
Support Tickets
Analytics
Analytics Dashboard
Daily Activity
Transaction Distribution
Monthly Volume Trend
Total Users
5
4 active
Total Transactions
58
Total Volume
$190410.00
Total Balance
$144760.00
Logout


Admin Panel
Admin Dashboard
Admin
Overview
Manage Users
All Transactions
Support Tickets
Analytics
Support Tickets
#1 - issue with budget and goal
From: Laiba Kanwal (bsse2480204@szabist.pk)
can not add any of them
28/06/2026, 21:46:30
in_progress
In Progress
Update
Open
In Progress
Resolved
Closed
Logout

Conclusion

The Digital Wallet Application (PakWallet System) successfully models, implements, and deploys a secure, production-ready 3-Tier framework for consumer financial activities. By grouping domain structures into normalized relational formats, the architecture eliminates redundant data hazards while keeping page performance light. The core ExpressJS middleware and asynchronous script integrations process high-volume user activity reliably, maintaining complete data accuracy across all entities. Testing across serverless database connections confirmed that the system operates with strict ACID compliance, verifying its viability as a scalable financial tech solution.

Future Improvements

While the present system covers all foundational requirements for asset security and multi-role operations, the architecture can scale to support additional real-world financial tech integrations in future versions:

- Multi-Currency Asset Wallets: Upgrading the schema with specialized exchange dictionaries to calculate multi-currency balances with real-time conversion monitoring.
- Two-Factor Authentication Protocols: Integrating time-based password verifications (TOTP via Google Authenticator) into the authentication filters to secure user sessions.
- Automated PDF Statement Exporters: Attaching server-side reporting utilities (such as PDFKit) to allow customers to generate and export certified transaction logs.
- Algorithmic Risk Alerts: Deploying machine learning categorization algorithms to identify unusual transaction frequencies, automatically flagging suspect transfers for administrative review.